Rootopia: Depth-Resolved Root Trait Analysis for Minirhizotrons and Flatbed Scans in R

Purpose, Standard Workflow, and Extensions

Johannes Cunow

Contents

Scope **2**

Summary . . . 2

Where Rootopia starts . . . 2

The package pipeline . . . 3

Part I: the standard minirhizotron workflow **4**

Stitch the frames, if there are several . . . 4

Hand the images to a segmenter . . . 4

The three image types . . . 5

Which image feeds which function . . . 8

Censor the rotation . . . 9

Depth, from geometry . . . 10

Insertion angle and the depth map . . . 11

Bin the depths . . . 13

Traits per depth bin . . . 14

Profile summaries . . . 16

The same profile over a folder . . . 18

Part II: extensions **19**

Root angle . . . 19

Rotation bias and circumferential pattern . . . 19

Comparing depth profiles . . . 22

Turnover between sessions . . . 25

Soil color, texture, and the rhizosphere . . . 26

Flatbed scans: the same traits without a depth axis . . . 30

Network shape and transport . . . 37

Reference **43**
What you can measure . . . 43
Per image, or per depth interval . . . 43
Per tube or scan . . . 43
Per segment, from the branching graph . . . 43
Per network . . . 44
Conventions . . . 44
Packages this builds on . . . 44
Where to go next . . . 45
References . . . 45
Software . . . 46

A printable version of this article is at Rootopia_Overview.pdf.

Scope

Summary

Rootopia turns segmented root images into root traits with a depth axis. It reads flatbed scans of washed roots and minirhizotron image sequences, and measures root length, diameter, surface, branching architecture, network topology, turnover between sessions, and soil color and texture. For a minirhizotron tube it converts pixel position into soil depth from the tube's diameter, insertion angle and scan resolution, so a trait can be reported per depth interval rather than per image.

Segmentation is not part of the package. Rootopia begins where a trained segmenter leaves off, and the first section below says exactly what it expects to be handed.

This article is the condensed reference: the scope, the image types and what consumes them, one complete worked profile, and pointers to the article that develops each remaining module. The algorithms and the edge cases live in How It Works.

Where Rootopia starts

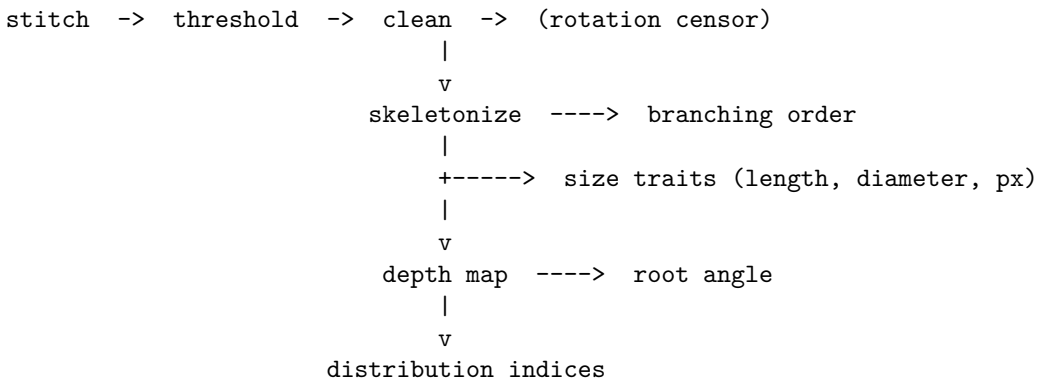
Roots are buried, and the two standard ways of imaging them each give up something. A flatbed scanner captures washed roots at high resolution under controlled light, which gives morphology but destroys the plant and loses the depth each root grew at. A minirhizotron keeps both the plant and the depth, by imaging soil through the wall of a transparent tube, but returns that tube's unrolled curved surface as a flat strip in which pixel position is not depth.

Finding roots in either image is a segmentation problem, and trained models solve it well enough to be used as given: RootDetector and RootPainter are the two this package is built around. What those tools do not do is turn a mask into root length density per 5 cm interval with a defensible depth axis. That is the gap Rootopia fills, and the boundary matters in both directions:

- **Nothing here segments an image.** `image_threshold()` exists for scans where roots really are brighter than their surroundings, such as washed roots on a dark tray, and a brightness threshold can do no more than that. Roots against soil need a trained model; on the bundled tube scan no threshold setting works, which is a property of the image rather than of the code.
- **Every conversion after the mask is a stated rule,** not a default to be inferred. Depth comes from measured geometry, lengths name their estimator, and each function names the image type it expects.

The package pipeline

Most analyses run left to right through this chain. You can enter at any point and stop at any point; nothing forces you through the whole thing.



Turnover and soil & color sit outside this chain: turnover compares two images, and soil & color works on the color channels rather than the root shape.

`root_depth_metrics()` is not a module. It is a batch wrapper that runs this chain over a folder of images.

Two patterns cover everything downstream of the depth map. A trait that reduces to a per-zone sum or mean goes through `terra::zonal()` in one call. A trait whose function needs a whole image, such as root length, goes through `depth_zoning()` one bin at a time, in a loop. Both appear below.

Part I: the standard minirhizotron workflow

This is the path almost every minirhizotron analysis takes, in order: censor the rotation, build a depth map, bin it, measure traits per bin, summarize the profile. Nothing is thinned on the fly – the segmentation and the skeleton both come from RootDetector, which is the hand-over described above, exercised. Part II then adds the modules that sit outside this path.

Stitch the frames, if there are several

Before anything else: a tube is often imaged as several overlapping frames, and a root crossing a frame boundary would otherwise be measured twice. If each tube is already one image, skip this. `stitch_root_scans()` groups files into tubes by a pattern in the filename, aligns consecutive frames by FFT phase correlation over a band along the overlapping edge, and blends them into one mosaic per tube.

```
# See what a folder holds, then pick a range in the same call
stitch_root_scans("path/to/scans", pattern = ".tiff", tubes = "ask")

res <- stitch_root_scans(
  "path/to/scans", pattern = ".tiff",
  blend      = "max",          # for segmented masks: never averages a root
  order_by   = "\\d+(?=\\.)",  # sort on the frame number, not the file name
  report     = TRUE
)
res$report      # tube | step | dx | dy | peak | overlap
res$report[order(res$report$peak), ][1:10, ]   # weakest joins first
```

Two settings decide whether a mosaic is usable. `blend = "max"` is the one for segmented masks: a linear feather *averages*, which turns a root pixel into a fractional value and ghosts thin roots wherever alignment is off by a pixel. And `edge_width` is the width of the band searched for the overlap, not the overlap itself, so it has to be at least as wide as the true overlap. The report's `peak` column is the correlation height at each join; sort by it to find the weak ones. See Image Stitching.

Hand the images to a segmenter

Stitched or not, the next step is not in this package. A scan has to be turned into a root mask by something trained to recognize roots, and there are three routes:

- **RootDetector** — a convolutional network for minirhizotron images. Returns the segmentation and a skeleton in one pass, and optionally the decay/production/constant coding that `root_turnover(method = "dpc")` reads.
- **RootPainter** — interactive training, where you correct the model as it learns. Returns a mask; use `skeletonize_image()` for the skeleton.
- **Anything else** that writes a binary mask. Rootopia reads PNG, JPEG, TIFF, BMP, `terra` and `raster` objects, `imager` images and plain arrays, so the format is rarely the obstacle.

Only one case does not need a trained model: washed roots photographed on a dark tray, where the roots really are brighter than the background, can be thresholded with `image_threshold()`. Roots against soil cannot — on the bundled tube scan no setting of `adaptive_k` separates them, which is a property of the image rather than of the code.

What comes back is up to three rasters per scan, and the rest of this article is about what to do with them.

The three image types

A segmenter hands back up to three rasters per scan, and they are not interchangeable.

Figures here use `zoom_plot()`, as the step-by-step vignettes do. These rasters are several thousand pixels wide, and shrinking one to fit a figure makes the graphics device drop the one-pixel-wide roots entirely; `zoom_plot()` draws a full-image overview with the magnified region outlined, plus that region at native resolution where thin features survive. `frac` sets the inset size, so the magnification is `1 / frac`.

```
# 3-band RootDetector output; layer 2 is the root channel.
data(seg_Oulanka2023_Session01_T067)
data(skl_Oulanka2023_Session01_T067)

dpi <- 150   # the scan resolution; every cm value below depends on it

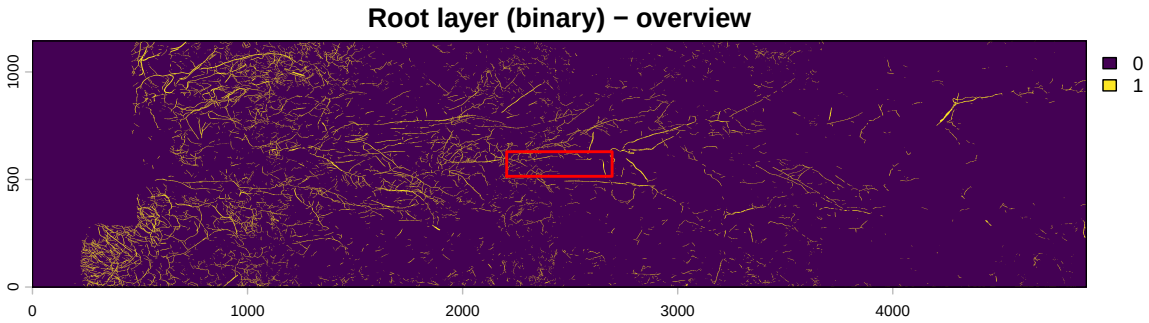
# Keep the full 3-band image (0/255) for the mask and depth steps below.
seg_img <- load_flexible_image(seg_Oulanka2023_Session01_T067,
                               output_format = "spatrast", scale = "none")

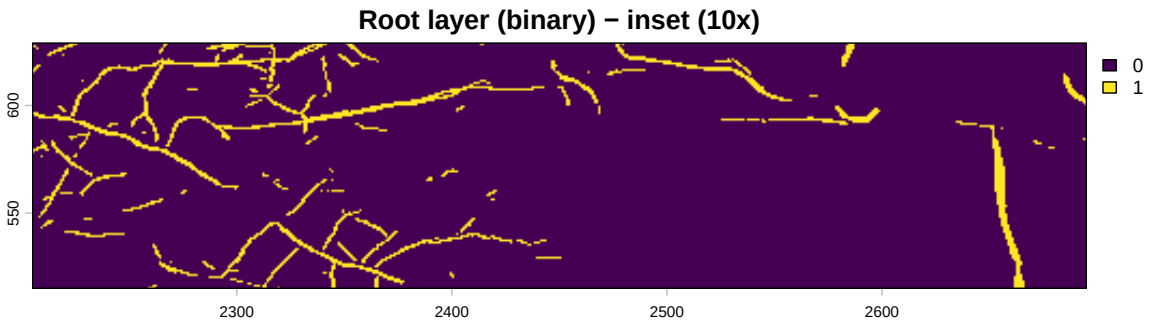
# load_flexible_image() also selects the root channel and binarizes it to 0/1
# in one call, so pixel sums and the void mask (abs(root_layer - 1)) are
# meaningful.
root_layer <- load_flexible_image(seg_Oulanka2023_Session01_T067,
                                  output_format = "spatrast",
                                  scale = "binary", select_layer = 2)

# The one-pixel center line, from the same RootDetector run.
skl <- load_flexible_image(skl_Oulanka2023_Session01_T067,
                           output_format = "spatrast",
                           scale = "binary", select_layer = 2)

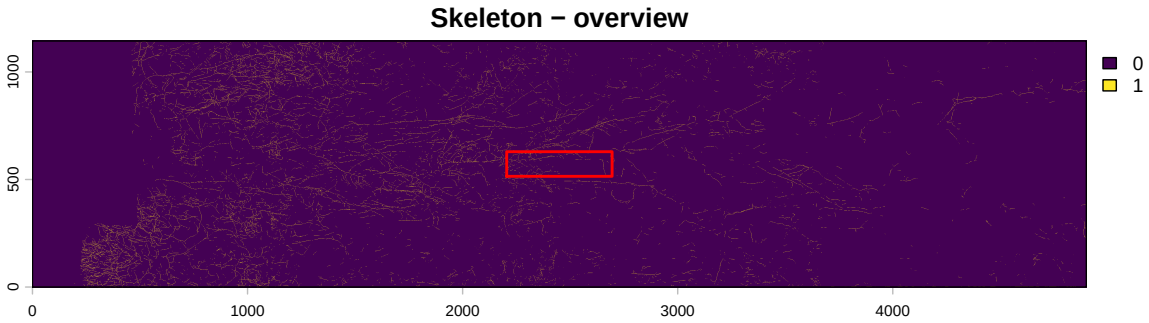
# RGB scan (a different session of the same tube, for the color demo)
data(rgb_Oulanka2023_Session03_T067)
rgb <- load_flexible_image(rgb_Oulanka2023_Session03_T067,
                           output_format = "spatrast", scale = "none")

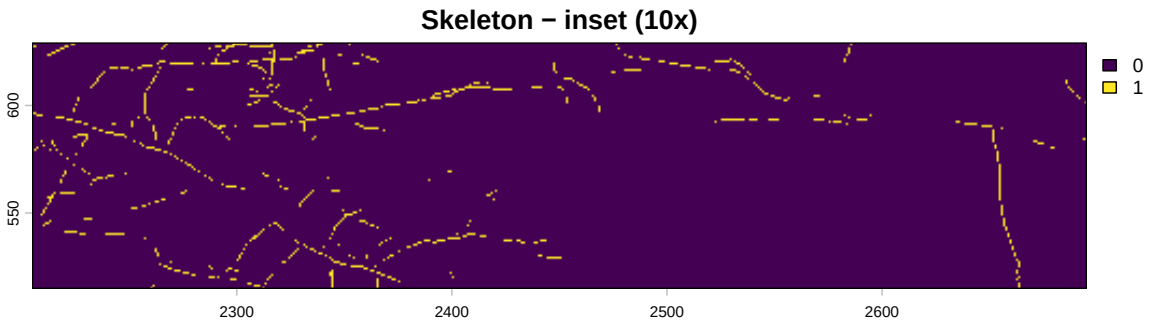
zoom_plot(root_layer, main = "Root layer (binary)", frac = 0.1)
```






```
zoom_plot(skl,        main = "Skeleton",           frac = 0.1)
```






```
zoom_plot(rgb,        main = "RGB scan",          frac = 0.1)
```

RGB scan − overview



RGB scan − inset (10x)



Each carries different information, so each feeds different functions.

Image	What it is	What it is for
RGB	the photograph	color, texture, soil class
Segmentation	solid root mask, 0/1	area, diameter, cleaning
Skeleton	one-pixel center line	length, tips, branching

RootDetector returns all three. RootPainter returns a mask, and `skeletonize_image()` thins it, so a missing skeleton is never a blocker.

Which image feeds which function

This is the table to keep. Passing the wrong type usually gives a wrong number rather than an error.

Function	Needs	Note
`count_pixels()`	segmentation	area, not length
`root_length()`	skeleton	`skeletonize = TRUE` thins a mask first
`branch_order_map()`	**both**	also where tips and forks come from
`root_diameter()`	**both**	mask for widths, skeleton for where
`network_metrics()`	segment table	from `branch_order_map()$edges`
`root_transport()`	segment table	needs diameters, so needs `mask =`
`root_scape_metrics()`	segmentation	patch structure of solid root (needs **landscapemetrics**)
`create_depthmap()`	any raster	reads the grid, not the values
`tube_coloration()`	RGB	three bands required
`classify_soil_rgb()`	RGB	per-pixel material class
`analyze_soil_texture()`	RGB	GLCM texture (needs **glcm**)
`root_turnover()`	two scans, or one DPC	per `tc_method`

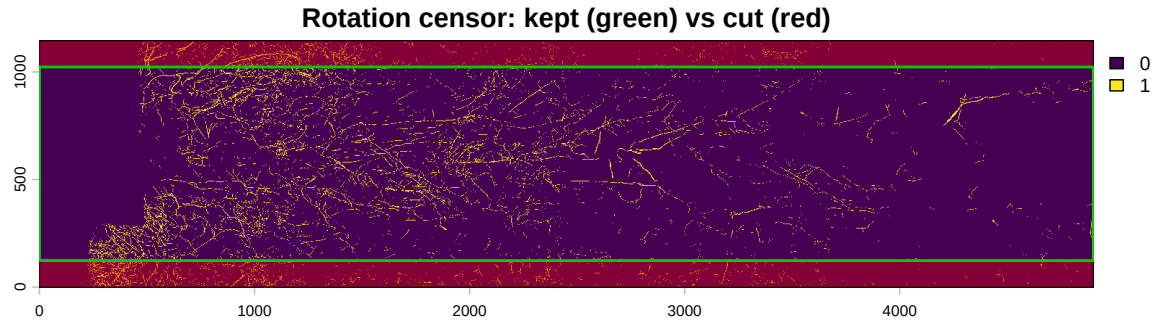
`root_diameter()` is the one to understand. Diameter comes from a distance transform of the *solid* mask: every root pixel is labeled with its distance to the nearest background pixel, which is the radius of the largest circle that fits inside the root there, and diameter is twice that. The skeleton then says *which* of those pixels to report, so each root contributes its center line once rather than its whole width. Pass only a skeleton and every diameter collapses to about one pixel. The same applies to `mask =` in the branching functions, where the continuation rule and the order-1 seed both depend on diameter.

Censor the rotation

A tube can turn between sessions, and the scanner does not cover a full circle, so roots near the edge of the scan are visible in one session and not the next. Left alone that is a systematic bias which looks like growth or death. Cropping every session to a common window of rows removes it, and it is standard procedure rather than an option.

```
# Center the window on the tube's rotational zero. estimate_rotation_center()
# finds it from the tape on the tube's upper side, once per tube; here the
# image midpoint stands in for it, so this article needs no extra packages.
r0 <- round(dim(root_layer)[1] / 2, 0)

censored <- rotation_censor(
  root_layer,
  center_offset  = r0,
  cut_buffer     = 0.02,
  fixed_width    = 900,    # rows kept; must fit inside the image height
  fixed_rotation = TRUE,   # equal width across sessions is what makes
  overlay        = TRUE,   # counts comparable
  main = "Rotation censor: kept (green) vs cut (red)"
)
```



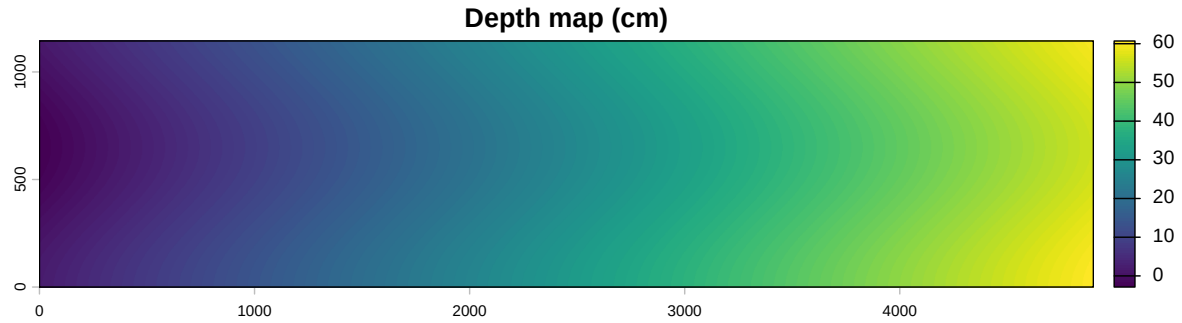
`fixed_rotation = TRUE` forces exactly `fixed_width` rows, which is what you want when comparing sessions: equal width is what makes counts comparable. `fixed_rotation = FALSE` instead cuts in proportion to a measured offset, so the output width varies between images. `estimate_rotation_shift()` measures that offset between two sessions by phase or cross correlation.

The rest of this article works on the uncensored image, so the depth figures cover the full circumference.

Depth, from geometry

```
depth_map <- create_depthmap(
  img                 = root_layer,
  geometry            = "tube",  # "window" planar, "flat" no depth axis
  tube_thicc          = 7,       # cm - measure your own tube
  angle_from_vertical = 45,      # degrees from vertical; 0 is upright
  dpi                 = dpi,
  start_soil          = 2.9,     # cm - from in-situ calibration
  center_offset       = 0.5      # tube top halfway down the image
)

terra::plot(depth_map, main = "Depth map (cm)")
```



Four measured quantities set this map, and each fails in its own way. `angle_from_vertical` is read from vertical, so 0 is an upright tube and 90 a horizontal one. `tube_thicc` is the inner diameter; if it or `dpi` is wrong the curvature wave cannot span the image and the function stops with `Wrong Tube Diameter`. `start_soil` is in centimeters and is subtracted from every depth, so an error in it shifts the whole profile by that amount and changes nothing else. `center_offset` rotates the curvature wave to where the top of your tube sits, as a fraction of one circumference.

Depth also runs along the image **width**. A scan with the soil surface at the top needs a quarter turn first; nothing errors if it is the wrong way round, the map is simply built along the wrong axis.

Insertion angle and the depth map

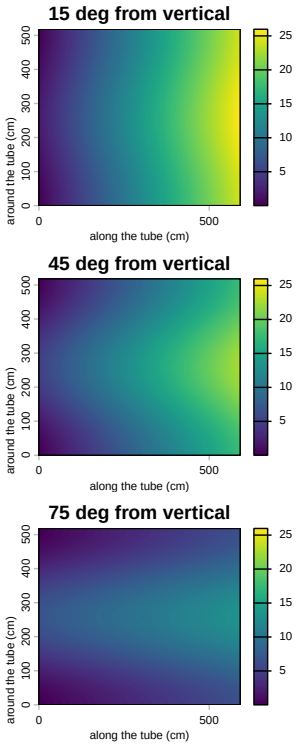
The angle is not a scaling factor. It splits a tube's depth between two axes: depth gained along the tube's length, and depth spread around its circumference. Leaning the tube over trades one for the other, and with `geometry = "tube"` both terms are visible in the same map.

```
# A 7 cm tube over 25 cm of its length: depth runs along the width, the
# circumference down the height, as in a real scan.
tube <- 7; sdpi <- 60
nr   <- round(tube * pi * sdpi / 2.54, 0) - 1          # one full turn
demo <- terra::rast(matrix(0, nrow = nr,
                           ncol = round(25 * sdpi / 2.54, 0)))

# Axes in centimeters rather than pixels, so the two directions can be read
# against each other.
terra::ext(demo) <- c(0, 25, 0, tube * pi)
angs <- c(15, 45, 75)

maps <- lapply(angs, function(a)
  create_depthmap(demo, geometry = "tube", tube_thicc = tube,
                  angle_from_vertical = a, dpi = sdpi, center_offset = 0))
lims <- range(vapply(maps, function(m) terra::minmax(m)[, 1], numeric(2)))
```

```
op <- graphics::par(mfrow = c(3, 1))
for (i in seq_along(angs))
  terra::plot(maps[[i]], range = lims, maxcell = Inf,
              xlab = "along the tube (cm)",
              ylab = "around the tube (cm)",
              main = sprintf("%d deg from vertical", angs[i]))
```




```
graphics::par(op)
```

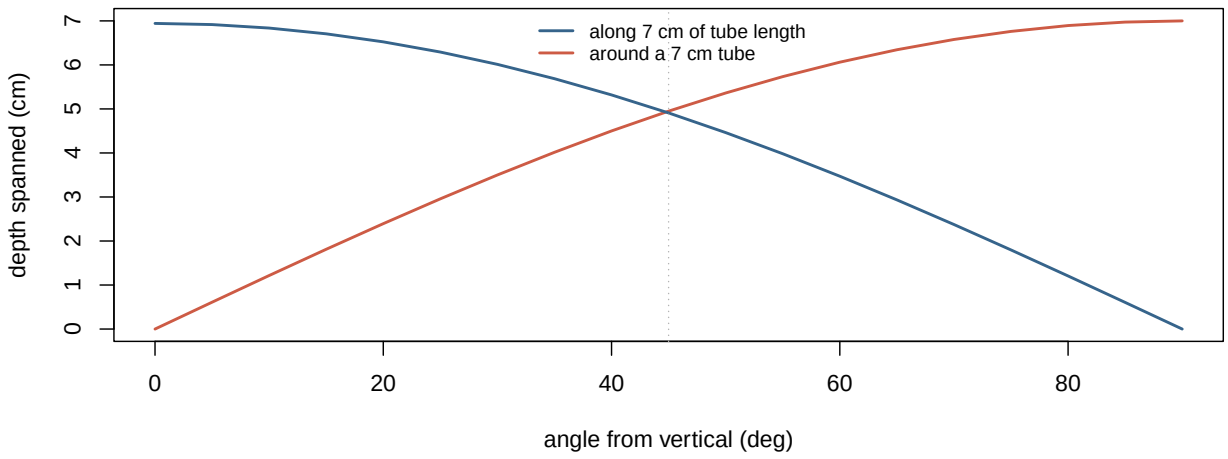
Measuring the two terms off the function needs one more thing: a length of tube to measure the depth gain over. Taking 7 cm of tube, the same as the diameter, makes the two terms two halves of one budget.

```
budget <- terra::rast(matrix(0, nrow = nr,
                             ncol = round(tube * sdpi / 2.54, 0)))

ang_seq <- seq(0, 90, by = 5)
terms <- vapply(ang_seq, function(a) {
  m <- terra::as.matrix(
    create_depthmap(budget, geometry = "tube", tube_thicc = tube,
                    angle_from_vertical = a, dpi = sdpi, center_offset = 0),
    wide = TRUE)
  c(circ = diff(range(m[, 1])), len = diff(range(m[1, ])))
}, numeric(2))

op <- graphics::par(mar = c(4, 4, 2, 1))
plot(ang_seq, terms["circ", ], type = "l", lwd = 2, col = "coral3",
     ylim = c(0, max(terms)), xlab = "angle from vertical (deg)",
     ylab = "depth spanned (cm)")
```

```
graphics::lines(ang_seq, terms["len", ], lwd = 2, col = "steelblue4")
graphics::abline(v = 45, col = "grey70", lty = 3)
graphics::legend("top", bty = "n", lwd = 2, cex = 0.8,
                 col = c("steelblue4", "coral3"),
                 legend = c("along 7 cm of tube length",
                            "around a 7 cm tube"))
```




```
graphics::par(op)
```

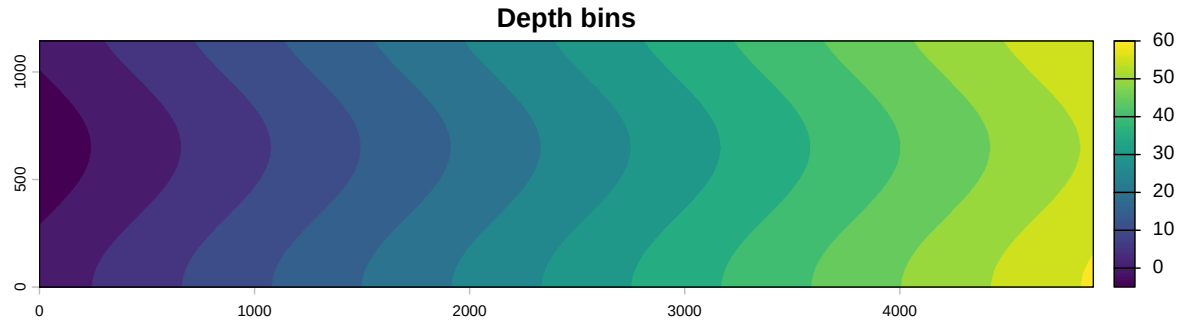
Both curves are depth spanned in centimeters over the same 7 cm of tube: 7 cm around the circumference, 7 cm along the length. That makes them two halves of one budget. They cross at 45 degrees at the same value, 4.95 cm, and at either end one of them holds all 7 cm while the other holds none.

Near upright, almost all of the depth is the left-to-right gradient along the tube and the circumference sits at nearly one depth, so the banding is faint. Near horizontal it is the reverse: the banding spans a full tube diameter while the length gradient flattens. The two curves cross at 45 degrees, the angle most minirhizotron work uses and the one angle at which the two terms cannot be told apart. With `geometry = "window"` the circumference term is dropped and only the length term remains; with `"flat"` both go, and every pixel shares one depth.

Bin the depths

```
depth_bins <- binning(depthmap = depth_map, nn = 5, round_option = "floor")

depths <- sort(unique(terra::values(depth_bins, mat = FALSE)))
depths <- depths[!is.na(depths) & depths >= 0]
depths
#>  [1]  0  5 10 15 20 25 30 35 40 45 50 55 60
terra::plot(depth_bins, main = "Depth bins")
```



`round_option = "floor"` gives the soil-science convention: the bin labeled 5 spans 5-10 cm and every bin is the full `nn` wide. The default, `"rounding"`, labels a bin by its center, so the bin labeled 0 spans -2.5 to +2.5 cm and is half width once negative depths are dropped. Per-bin densities are unaffected either way, because they divide by each bin's own measured area, but any total that multiplies a per-bin value by `nn` is biased by that half-width bin.

Traits per depth bin

Pixel counts reduce to a per-zone sum, so the whole profile comes out of one call.

```
root_px <- terra::zonal(root_layer, depth_bins, "sum", na.rm = TRUE)
void_px <- terra::zonal(abs(root_layer - 1), depth_bins, "sum", na.rm = TRUE)
colnames(root_px) <- c("depth", "rootpx")
colnames(void_px) <- c("depth", "voidpx")

depth_data <- merge(root_px, void_px, by = "depth")
depth_data <- depth_data[depth_data$depth >= 0, ]
depth_data$area_cm2 <- (depth_data$rootpx + depth_data$voidpx) *
                       (2.54 / dpi)^2
head(depth_data, 4)
#>   depth rootpx voidpx area_cm2
#> 2     0   6200 430997 125.3609
#> 3     5  48416 429355 136.9950
#> 4    10  43035 434635 136.9660
#> 5    15  27884 449851 136.9847
```

Root length does not reduce that way. Kimura's estimator needs a whole skeleton, and it is not additive: the depth bins do not add up to the whole-image value. That is correct for a non-linear estimator rather than a leak, and it is why the trait is computed one bin at a time.

```
length_by_depth <- do.call(rbind, lapply(depths, function(d) {
  skl_d <- depth_zoning(skl, depth_map = depth_bins, depth = d)
```

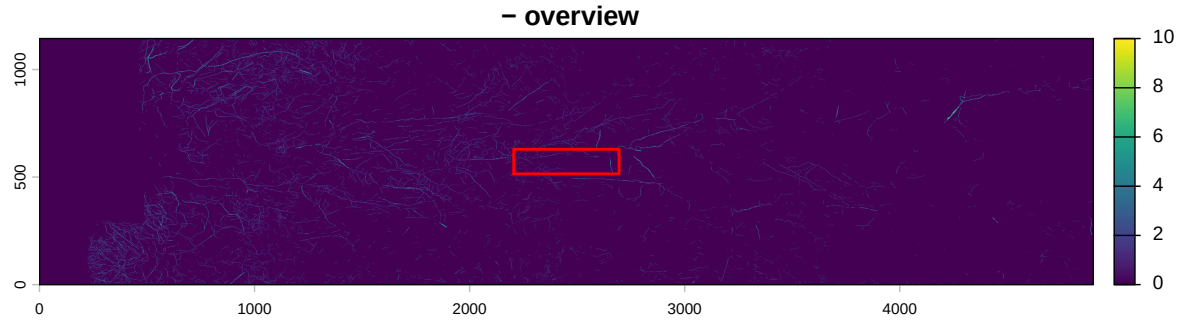
```
  data.frame(depth      = d,
             rootlength = root_length(skl_d, unit = "cm", dpi = dpi,
                                      show_messages = FALSE))
}))

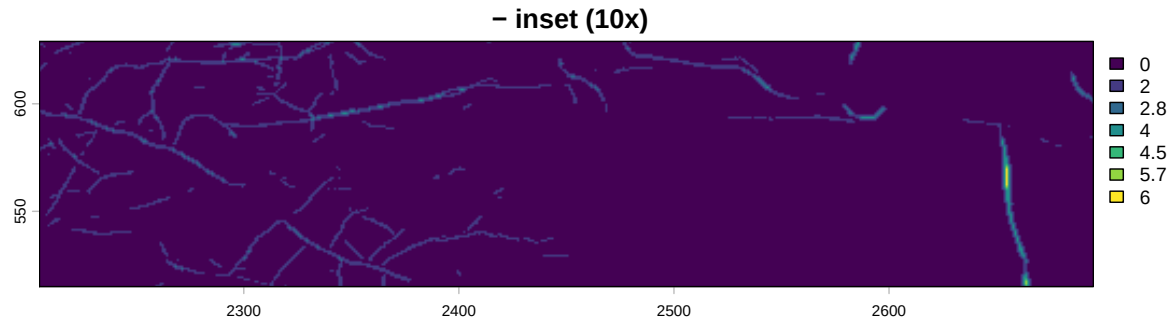
depth_data <- merge(depth_data, length_by_depth, by = "depth")
depth_data$rootlength.density <- depth_data$rootlength / depth_data$area_cm2
head(depth_data, 4)
#>   depth rootpx voidpx area_cm2 rootlength rootlength.density
#> 1     0   6200 430997 125.3609   33.89568           0.2703848
#> 2     5  48416 429355 136.9950  231.56465           1.6903146
#> 3    10  43035 434635 136.9660  217.51774           1.5881145
#> 4    15  27884 449851 136.9847  137.14314           1.0011568
```

Diameter needs both images at once: the mask gives the width, the skeleton says which pixels to report. `root_diameter()` returns the local thickness across the whole scan as well as the per-skeleton-pixel diameters, so it is worth looking at before reducing it to a per-interval mean.

```
diam_result <- root_diameter(
  root_layer,
  skeleton_img = skl,
  unit         = "cm",
  dpi          = dpi,   # not optional: the default is 300
  select_layer = NULL
)

# root thickness
zoom_plot(diam_result$distance_map_rast, frac = 0.1)
```





Diameter is a per-pixel value, so it reduces with `zonal()` like the counts, once its raster is snapped onto the depth grid.

```
terra::ext(diam_result$diameter_rast) <- terra::ext(depth_bins)

diam_by_depth <- terra::zonal(diam_result$diameter_rast, depth_bins,
                              "mean", na.rm = TRUE)
colnames(diam_by_depth) <- c("depth", "avg.diameter")

depth_data <- merge(depth_data, diam_by_depth, by = "depth")
head(depth_data, 4)
#>   depth rootpx voidpx area_cm2 rootlength rootlength.density avg.diameter
#> 1     0   6200 430997 125.3609   33.89568           0.2703848   0.03808811
#> 2     5  48416 429355 136.9950  231.56465           1.6903146   0.03951835
#> 3    10  43035 434635 136.9660  217.51774           1.5881145   0.03738350
#> 4    15  27884 449851 136.9847  137.14314           1.0011568   0.03656560
```

Profile summaries

```
# Mean rooting depth - depth-weighted mean
ok      <- depth_data[!is.na(depth_data$rootlength.density) &
                      depth_data$rootlength.density > 0, ]
mrd_val <- MRD(w = ok$depth, roots = ok$rootlength.density)

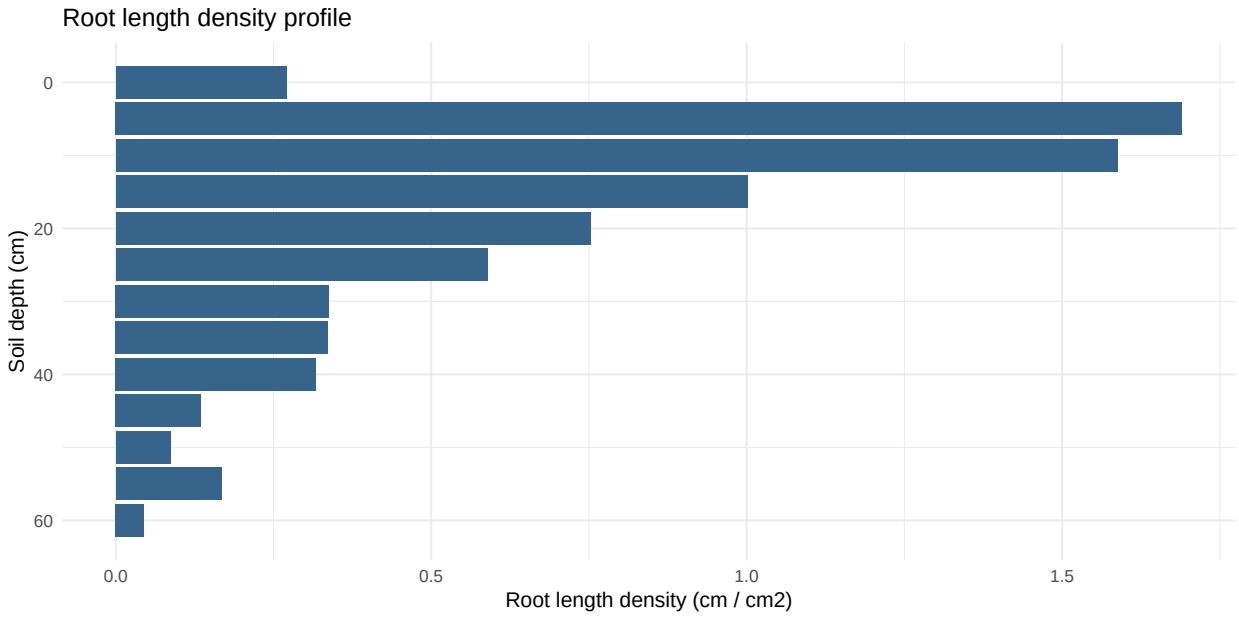
total_ld <- sum(depth_data$rootlength.density * 5, na.rm = TRUE)

cat(sprintf("MRD: %.2f  Total length density: %.4f\n", mrd_val, total_ld))
#> MRD: 17.23  Total length density: 36.5845
```

`MRD()` stops on `NA`, on negative root values, and on an all-zero profile rather than returning a number built from part of the data, which is why the profile is filtered first and the filtering stays visible.

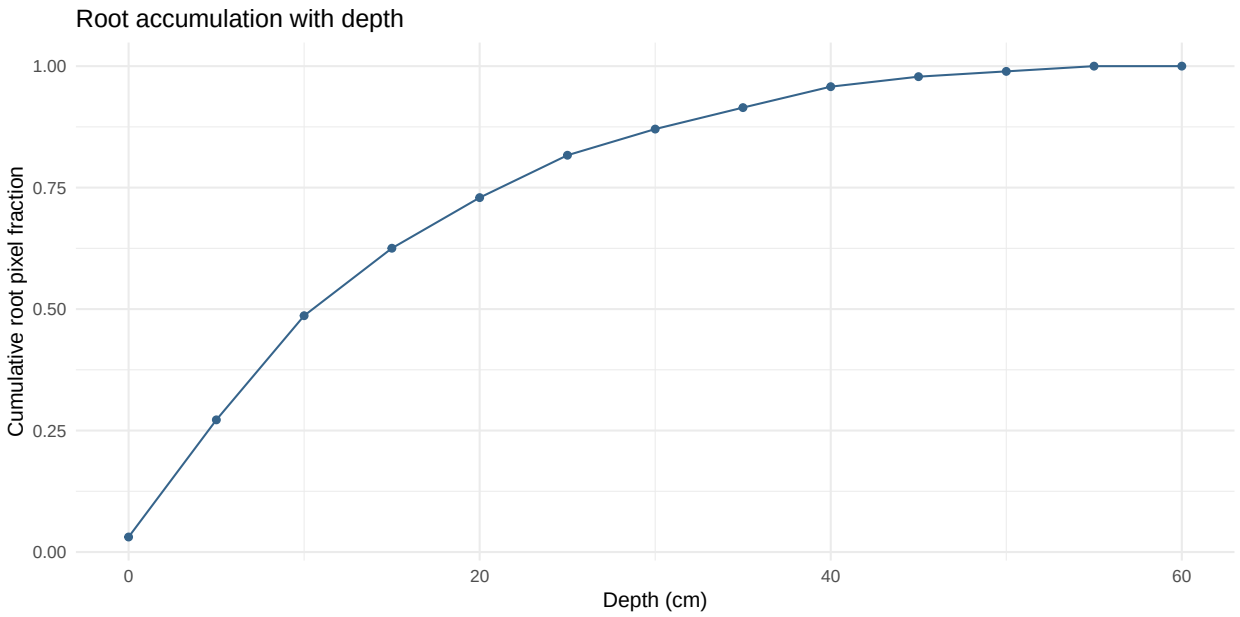
```r
library(ggplot2)

ggplot(depth_data, aes(x = depth, y = rootlength.density)) +
  geom_col(fill = "steelblue4") +
  coord_flip() +
  scale_x_reverse() +
  theme_minimal() +
  labs(x = "Soil depth (cm)", y = "Root length density (cm / cm2)",
       title = "Root length density profile")
```




```r
depth_data$Tube <- "T067"
depth_data$rootpx.cumulative <- root_accumulation(
  depth_data,
  group    = "Tube",
  depth    = "depth",
  variable = "rootpx",
  stdrz    = "additive"   # "counts" (raw), "additive" (/max), "relative" (/sum)
)

ggplot(depth_data, aes(x = depth, y = rootpx.cumulative)) +
  geom_line(color = "steelblue4") +
  geom_point(color = "steelblue4") +
  theme_minimal() +
  labs(x = "Depth (cm)", y = "Cumulative root pixel fraction",
       title = "Root accumulation with depth")
```



`stdrz` picks the denominator and only one option ends at 1. `"additive"` divides the cumulative curve by its own maximum, so curves from tubes with very different total abundance compare by shape alone. `"counts"` is the raw cumulative sum, and `"relative"` divides by the sum of the cumulative values, which is a different scale.

The same profile over a folder

```
result <- root_depth_metrics(
  path_seg            = "scans/segmented/2023_06/",
  path_skl            = "scans/skeleton/2023_06/",
  tube_names          = ids,      # one unique id per file
  dpi                 = 150,
  geometry            = "tube",
  insertion_angles    = 45,       # degrees from vertical
  tube_diameter_cm    = 7,
  soil_starts         = 2.9,      # cm
  depth_interval_cm   = 5,
  bin_round           = "floor",
  session             = "2023_06"
)
```

`path_skl` is optional; left out, skeletons are thinned internally. Branching order, root angle, color and landscape metrics are separate `calc_*` switches, all off by default because each costs real time. Batch Processing covers which arguments matter for which geometry.

Part II: extensions

Part I is the standard path. Everything here sits beside it: another trait from the same scan, a second session, a different imaging geometry, or a step that comes before analysis. The order runs from closest to that path to furthest from it. Each section names the article that develops it.

Root angle

`deep_drive()` asks one question of a depth map and a skeleton: what fraction of root pixels head in the direction that would take them deeper fastest? It derives the direction each root pixel is actually heading from a D8 flow-direction pass over the depth map, compares it against which of the eight neighbors lies deepest, and returns the fraction that agree.

```
dd <- deep_drive(DepthMap = depth_map, RootMap = skl)

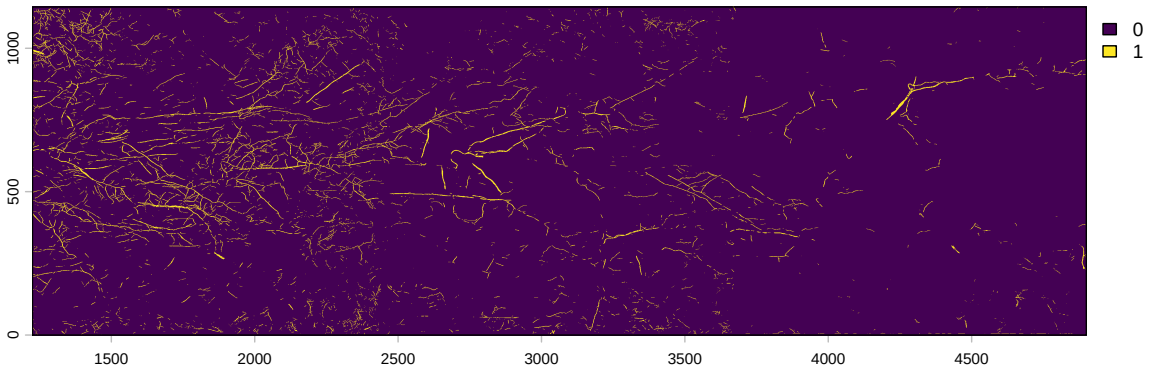
cat(sprintf("deep_drive: %.3f of root pixels head the deepest way\n", dd))
#> deep_drive: 0.273 of root pixels head the deepest way
```

One number between 0 and 1, so it compares across tubes. There is no partial credit: a pixel heading 45 degrees from optimal counts the same as one heading 180 degrees away, which makes it a measure of agreement rather than of deviation. It needs a real depth gradient, so it means nothing on a flatbed scan. The batch wrapper reports it as `deep_drive` alongside `mean.steepness.angle`.

Rotation bias and circumferential pattern

Part I censored the rotation. The same slicing answers a second question: are roots spread evenly around the tube, or concentrated on one side? `slice_rotation()` cuts the image into bands of rows in rotation order, and a trait measured per band becomes a sequence indexed by circumferential position.

```
# Trim the shallowest quarter of the depth axis (columns), where the tape and
# the noisiest soil sit. The rotation axis is left whole, since that is what the
# slices below are cut along.
e <- terra::ext(root_layer)
rot_layer <- terra::crop(root_layer,
                         terra::ext(e[1] + e[2] / 4, e[2], e[3], e[4]))
terra::plot(rot_layer, maxcell = Inf)
```




```r
# create circumferential zones
n_slices <- 48
slices <- slice_rotation(rot_layer, n_slices)

# tallies from the binary mask: roots = 1, soil/background = 0
n_root <- function(s) {
  v <- terra::values(s, mat = FALSE); sum(v > 0, na.rm = TRUE)
}
n_soil <- function(s) {
  v <- terra::values(s, mat = FALSE); sum(v == 0, na.rm = TRUE)
}

slice_traits <- data.frame(
  slice   = seq_len(n_slices),
  rootlen = sapply(slices, root_length, unit = "cm", dpi = dpi,
                   show_messages = FALSE),  # cm
  rootpx  = sapply(slices, n_root),
  soilpx  = sapply(slices, n_soil)
)

# root length per observed area (root + soil pixels)
slice_traits$rld <- slice_traits$rootlen /
                    ((slice_traits$rootpx + slice_traits$soilpx) *
                       (2.54 / dpi)^2)
```

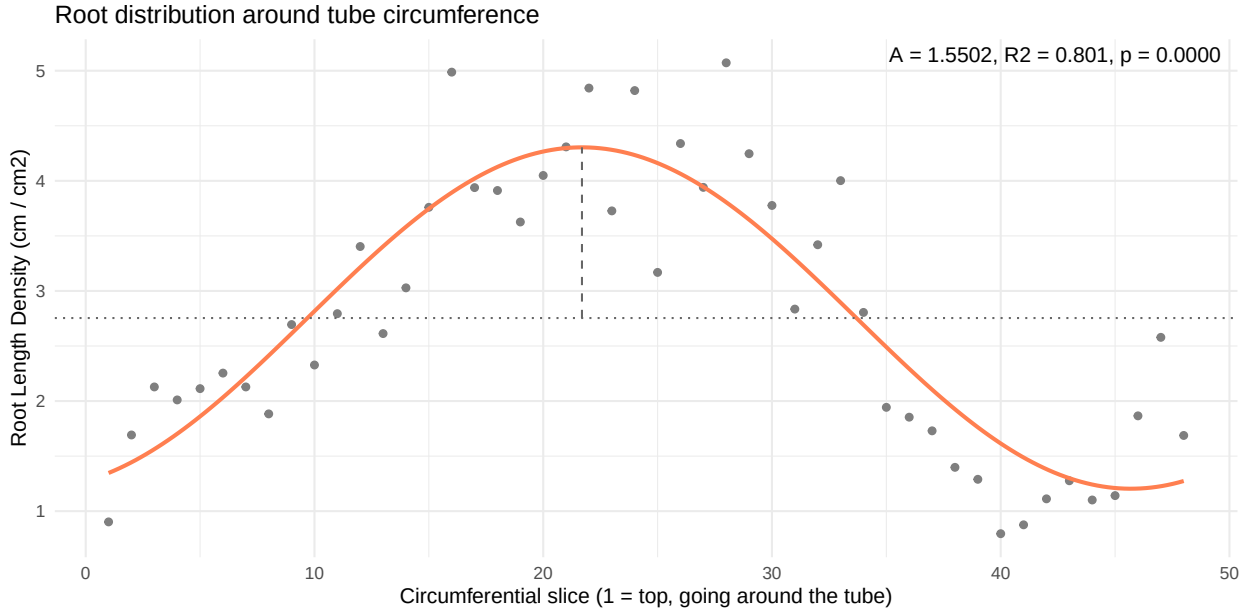
`rhythmicity()` fits `amplitude * sin(2*pi/period * (x + phase)) + offset` and tests whether the amplitude differs from zero. The fit is a Levenberg-Marquardt least squares solve from **minpack.lm**, and the amplitude tests follow the likelihood-based rhythmicity tests of diffCircadian (Ding *et al.* 2021): `method = "F"`, `"LR"` and `"Wald"` are the three tests that package distinguishes. Fixing the period to the number of slices means one full turn of the tube, so a significant amplitude says root abundance varies systematically with circumferential position. Nothing in the fit is specific to time: this is a spatial use of a tool normally introduced for time series, and `res$peakTime` is a slice index rather than a time.

```r
res <- rhythmicity(
  x          = slice_traits$slice,
  y          = slice_traits$rld,
  fix_period = n_slices,
  method     = "F"
)

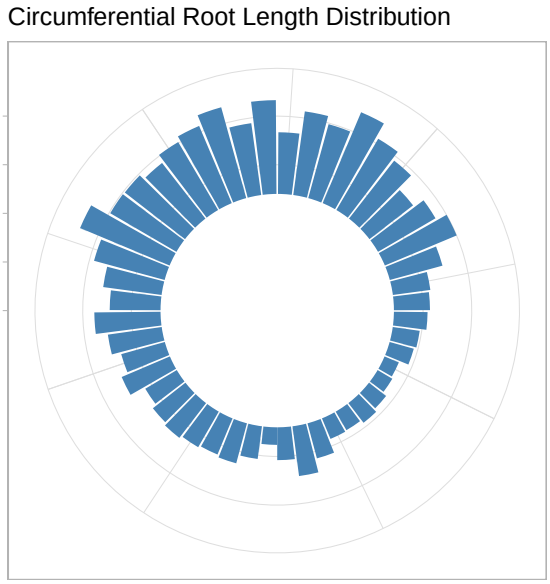
cat(sprintf("Mean %.4f | Amplitude %.4f | Phase %.2f | R2 %.3f | p %.4f\n",
            res$offset, res$amplitude, res$phase, res$R2, res$p_value))
#> Mean 2.7540 | Amplitude 1.5502 | Phase 38.31 | R2 0.801 | p 0.0000
```

```r
# Smooth fitted sine over the slice index
fit_df <- data.frame(slice = seq(1, n_slices, length.out = 200))
fit_df$rld <- res$amplitude *
              sin(2 * pi / n_slices * (fit_df$slice + res$phase)) +
              res$offset

ggplot(slice_traits, aes(slice, rld)) +
  geom_point(color = "gray50") +
  geom_line(data = fit_df, color = "coral", linewidth = 1) +
  geom_hline(yintercept = res$offset, color = "gray40", linetype = 3) +
  annotate("segment", x = res$peakTime, xend = res$peakTime,
           y = res$offset, yend = res$offset + res$amplitude,
           linetype = 2, color = "gray40") +
  annotate("text", x = Inf, y = Inf, hjust = 1.05, vjust = 1.5,
           label = sprintf("A = %.4f, R2 = %.3f, p = %.4f",
                           res$amplitude, res$R2, res$p_value)) +
  theme_minimal() +
  labs(x = "Circumferential slice (1 = top, going around the tube)",
       y = "Root Length Density (cm / cm2)",
       title = "Root distribution around tube circumference")
```



```r
# cross section view
inner <- 6
ggplot(slice_traits, aes(slice, rld + inner)) +
  geom_col(fill = "steelblue") +
  coord_polar(start = pi) +
  xlab("") + ylab("") +
  ggtitle("Circumferential Root Length Distribution") +
  theme_light() +
  theme(axis.text = element_blank()) +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 0, ymax = inner,
           fill = "white", color = NA)
```



A significant, non-trivial amplitude means more root material on one side of the tube than the other – from gravitropism, from light reaching one face of the installation, or from the installation angle itself. `estimate_rotation_center()` and `estimate_rotation_shift()` locate the rotational zero and the offset between sessions. See Rotation Bias.

Comparing depth profiles

Two profiles can differ in two ways that are worth keeping apart: the same root sitting at a different depth, or a different amount of root at the same depth. The earth-mover idea measures the first. **Distance here means depth**: how far down the profile the root mass would have to be carried, in centimeters, to turn one profile into the other. A profile shifted 35 cm costs more than one shifted 10 cm, and a profile of a different shape costs whatever rearranging it takes.

In one dimension that has an exact picture. Plot each profile as the cumulative share of total root length from the surface down, and the area between the two curves *is* that distance. At any depth the horizontal gap between the curves is the share of root that has to cross that depth; adding it up down the profile gives the total, in centimeters.

```r
z  <- seq(0, 60, by = 5)                    # 13 intervals, 5 cm each
```

```r
# A surface-weighted reference, with no mass below 25 cm. Keeping the support
# in the upper half of the grid matters: a profile shifted past the bottom of
# the grid loses the mass that falls off, and renormalizing what is left
# changes its shape, so it is no longer the same profile moved down.
p0 <- exp(-abs(z - 10) / 7)
p0[z > 25] <- 0
reference <- p0 / sum(p0)

# Move the whole profile down by k intervals.
shift_by <- function(p, k) c(rep(0, k), head(p, length(p) - k))

shifted10 <- shift_by(reference, 2)         # 2 intervals = 10 cm
shifted35 <- shift_by(reference, 7)         # 7 intervals = 35 cm
even      <- rep(1 / length(z), length(z))  # equal root in every interval

# A two-layer profile: a surface mode and a second one at 45 cm. The weight on
# the deep mode is solved so this profile has exactly the reference's mean
# rooting depth -- same center of mass, different shape.
shallow <- exp(-z / 5);           shallow[z > 15] <- 0
deep    <- exp(-abs(z - 45) / 7); deep[z < 30]    <- 0
shallow <- shallow / sum(shallow)
deep    <- deep / sum(deep)

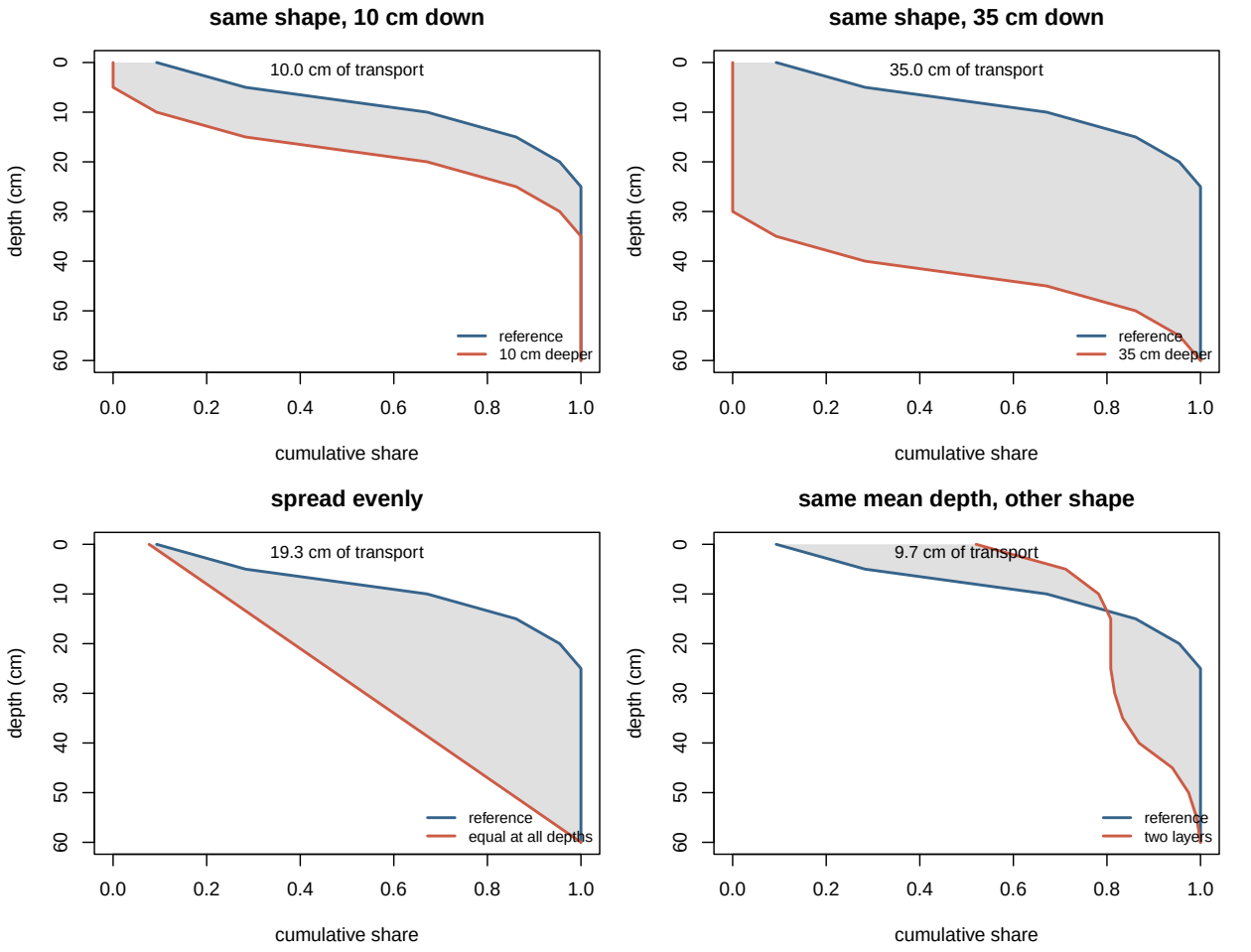
mean_depth <- function(p) sum(z * p)
w <- (mean_depth(reference) - mean_depth(shallow)) /
     (mean_depth(deep)      - mean_depth(shallow))
bimodal <- (1 - w) * shallow + w * deep

# Distance in centimeters of depth: the area between the two cumulative
# curves, measured down the depth axis. This is what
# compare_depth_distribution(metric = "wasserstein") returns.
emd_cm <- function(a, b) sum(abs(cumsum(a) - cumsum(b))) * diff(z)[1]

compare_panel <- function(b, label_b, title) {
  Fa <- cumsum(reference)
  Fb <- cumsum(b)
  plot(NA, xlim = c(0, 1), ylim = rev(range(z)),
       xlab = "cumulative share", ylab = "depth (cm)", main = title)
  graphics::polygon(c(Fa, rev(Fb)), c(z, rev(z)), col = "grey88",
                    border = NA)
  graphics::lines(Fa, z, col = "steelblue4", lwd = 2)
  graphics::lines(Fb, z, col = "coral3",     lwd = 2)
  graphics::legend("bottomright", bty = "n", lwd = 2, cex = 0.8,
                   col = c("steelblue4", "coral3"),
                   legend = c("reference", label_b))
  graphics::mtext(sprintf("%.1f cm of transport", emd_cm(reference, b)),
                  side = 3, line = -1.3, cex = 0.75)
}

op <- graphics::par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
compare_panel(shifted10, "10 cm deeper",      "same shape, 10 cm down")
compare_panel(shifted35, "35 cm deeper",      "same shape, 35 cm down")
compare_panel(even, "equal at all depths",    "spread evenly")
```

```
compare_panel(bimodal, "two layers",          "same mean depth, other shape")
```




```
graphics::par(op)
```

The first two panels hold the same root in the same shape and differ only in how far down it sits, so the band widens in proportion – and they are the check on the metric: a profile moved down by a fixed distance has to return that distance, and 10 cm and 35 cm are what the shaded areas measure. The third holds the same amount of root in every interval instead of peaking near the surface, so its cumulative curve is a straight line: the band is a different shape rather than simply wider, because the mass has to be spread out as well as moved. There is no shift to report for it, and its 19.3 cm is what rearranging one profile into the other costs.

The fourth panel is the one to read twice. It holds a two-layer profile – a surface mode and a second one at 45 cm – with the weight on the deep mode solved so that its mean rooting depth is exactly the reference's.

```
c(reference = mean_depth(reference), bimodal = mean_depth(bimodal))
#> reference   bimodal
#>  10.68323  10.68323
emd_cm(reference, bimodal)
#> [1] 9.669479
```

`MRD()` reports no difference at all between these two stands. The transport distance is 9.7 cm, and every centimeter of it is shape.

The curves show why. In the first three panels they never cross: one profile is deeper than the other at every depth, all the root travels the same way, and the distance comes out equal to the difference in mean rooting depth – 10, 35 and 19.3 cm. In the fourth they cross, because the bimodal profile holds more root than the reference near the surface and more again at depth. Mass has to move up as well as down; the two directions cancel in the mean and add in the distance. Crossing cumulative curves are the visual test for a difference that mean rooting depth cannot see.

`compare_depth_distribution()` is the function for this, with `metric = "wasserstein"` (default), `"js"` or `"kl"`, and optional tail weighting so deep layers can count for more than shallow ones. Jensen-Shannon and Kullback-Leibler compare bin by bin and have no notion that one bin is adjacent to another, so two profiles that do not overlap score much the same whether they are 10 cm or 50 cm apart.

The shaded areas above and the function agree: with `index` giving the depth of each bin, `compare_depth_distribution()` returns the mass-weighted mean distance the root has to travel, so a profile shifted 10 cm returns 10. Interpreting it needs two things kept in mind. The profiles are normalized, so total root amount divides out – a tube with ten times the root but the same depth distribution scores 0, and abundance has to be reported separately. And the value is bounded below by the difference in mean rooting depth: two tubes 3 cm apart in `MRD()` are at least 3 cm apart here. That bound is tight whenever the cumulative curves do not cross, which is the ordinary case for a profile that simply deepened. What is informative is the gap: a distance well above the difference in mean depth means root moved both up and down, so the shapes differ and not just the center of mass. The two-layer panel above is that case taken to its limit – the difference in mean depth is zero and the whole distance is shape.

Turnover between sessions

Two methods, for two kinds of input, answering different questions.

```
data(skl_Oulanka2023_Session03_T067)
t1 <- load_flexible_image(skl_Oulanka2023_Session01_T067,
                          output_format = "spatrast")
t2 <- load_flexible_image(skl_Oulanka2023_Session03_T067,
                          output_format = "spatrast")

root_turnover(img1 = t1, img2 = t2, method = "tc", tc_method = "kimura",
              dpi = dpi, unit = "cm")
#>   standingroot_t1 standingroot_t2 production newroot.per_t1 newroot.per_t2
#> 1        17875.84        16835.23  -1040.613        -0.0582        -0.0618
```

Temporal comparison measures root amount in each image and takes the difference, so it sees only the *net* change: root that grew and died between the two scans is invisible to it.

```
data(TurnoverDPC_data)
dpc <- load_flexible_image(TurnoverDPC_data, output_format = "spatrast",
                           scale = "none")
root_turnover(dpc, method = "dpc")
#>     tape constant production   decay newgrowth.ratio decay.ratio constant.ratio
#> 1 887012   438681    2770138 3355722          0.8633      0.8844         0.0668
```

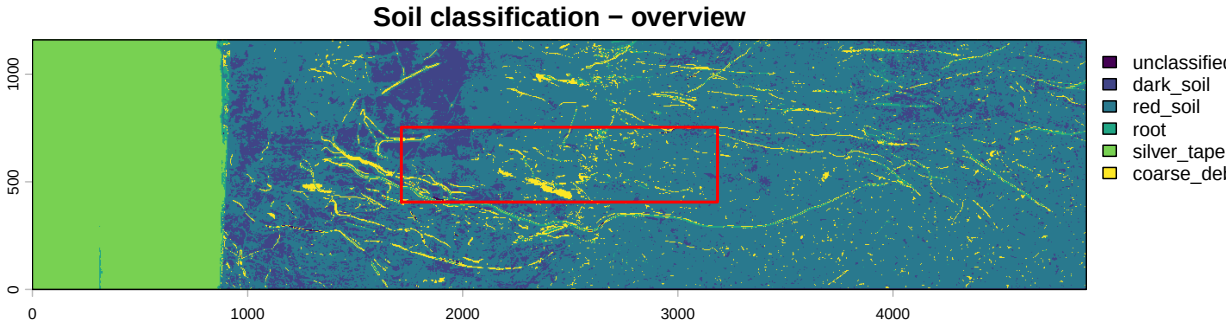
Decay-production-constant decomposes a single three-layer image in which a segmenter has already color-coded each pixel as produced, decayed or unchanged, which resolves *gross* change instead. `include_virtualroots` switches the denominator between "present at this time point" and "present at any time point", and the choice changes the ratios. See Special Topics.

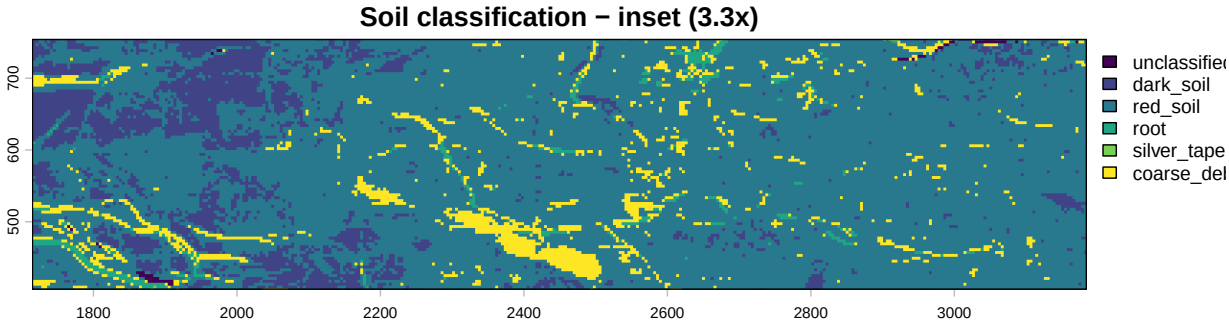
Soil color, texture, and the rhizosphere

Everything that reads the color channels rather than the root shape. `classify_soil_rgb()` assigns every pixel to a material class by nearest-centroid matching in CIE LAB, leaving pixels too far from any centroid unclassified rather than forcing them into the nearest class.

```
# downsample_fact speeds up the demo; drop it for full resolution
result <- classify_soil_rgb(rgb, downsample_fact = 4, verbose = FALSE)

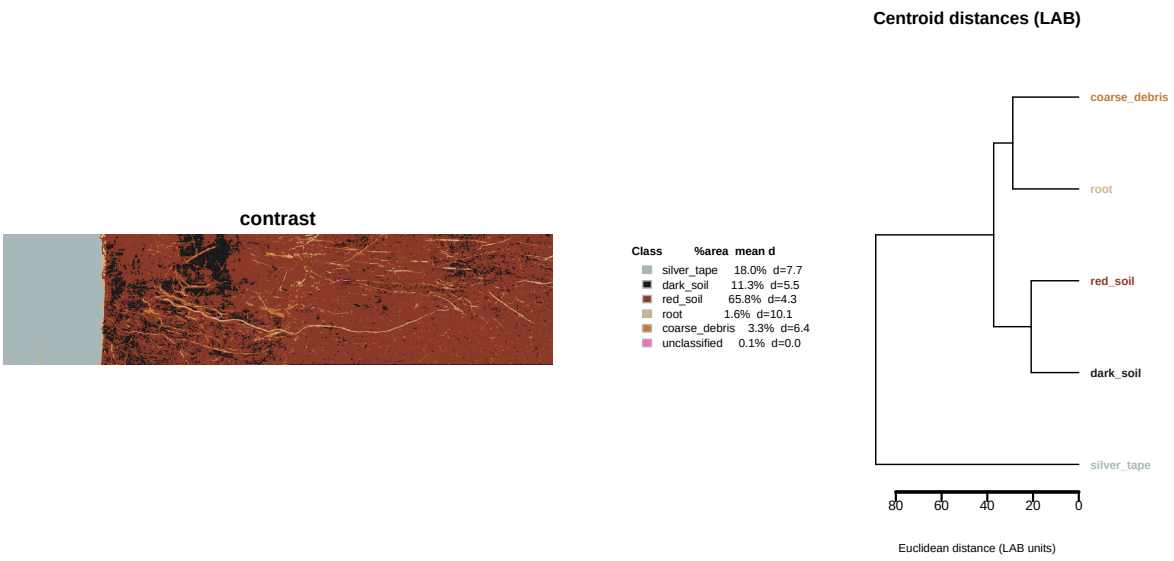
# A SpatRaster of class IDs (factor levels are the class names)
zoom_plot(result$map, main = "Soil classification")
```





```
result$metrics
#>           class n_pixels pct_pixels L_mean A_mean B_mean L_sd A_sd B_sd R_mean
#> 1     dark_soil    40252     11.331    8.8    1.5    2.4 3.38 1.45 1.35   28.3
#> 2      red_soil   233641     65.768   13.9    4.4    5.9 3.55 1.99 2.10   44.2
#> 3          root     5707      1.606   34.5    4.6   10.7 8.19 3.26 5.74   94.7
#> 4   silver_tape    63821     17.965   73.1   -2.0    0.9 3.63 0.55 1.06  176.4
#> 5 coarse_debris    11632      3.274   22.5    7.9   11.9 3.57 2.96 3.39   70.4
#> 6  unclassified      197      0.055   20.1   18.6   22.7 7.00 2.11 4.84   80.2
#>   G_mean B_mean_rgb mean_dist_to_centroid hex_actual
#> 1   23.9       20.9                  5.51    #1C1816
#> 2   33.0       27.1                  4.32    #2C211B
#> 3   78.4       64.6                 10.11    #5F4E40
#> 4  180.6      177.8                  7.67    #B0B5B2
#> 5   49.1       36.5                  6.38    #473124
#> 6   36.5       13.9                    NA    #50240F
```

```
plot_soil_classification(result)
```

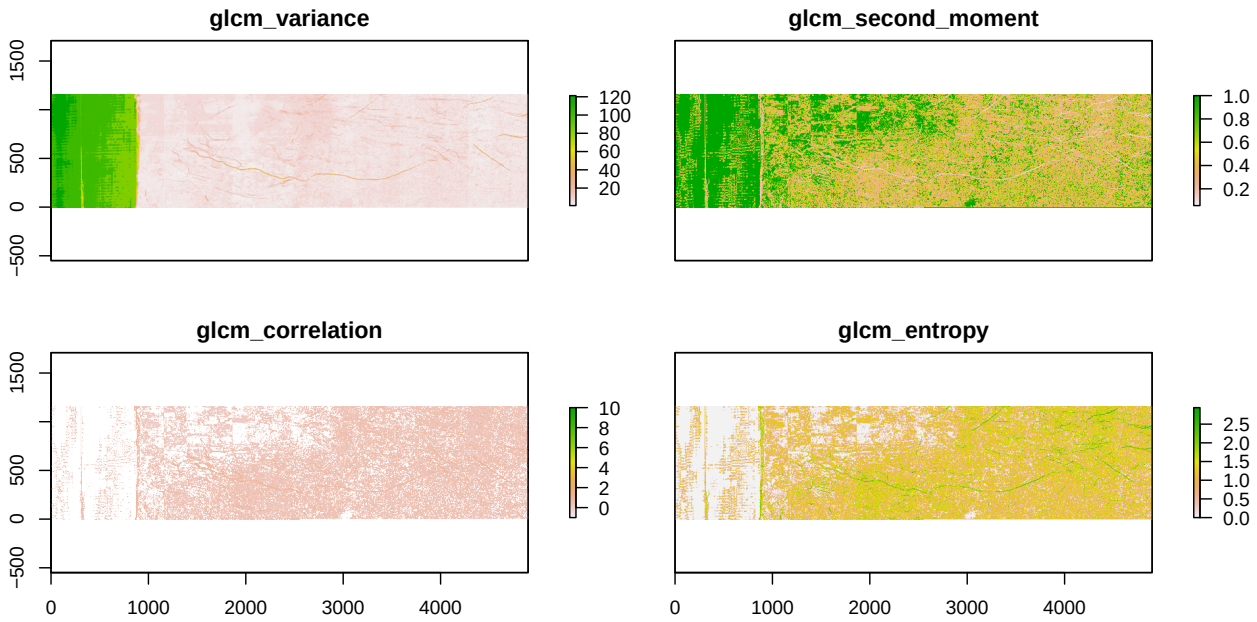


The returned list carries per-class statistics: pixel counts, area fractions, mean LAB and RGB colors, and the mean distance to the centroid. Watch the unclassified fraction – a high value means the shipped centroids do not match your scanner, and `build_soil_centroids()` derives your own from color picks. `plot_soil_classification()` is the figure to read it from: the class map, a legend in dendrogram order, and a dendrogram of inter-class LAB distances, so you can see which classes sit close enough together to be confused. `color_mode` switches the palette: `"contrast"` is the default and the one used here, `"vibrant"` saturates it, and `"centroid"` paints each class in its own measured mean color – which looks faithful but puts several soil classes in nearly the same red-brown, so it is the hardest of the three to read.

Texture is delegated: `analyze_soil_texture()` converts the band to a `raster` object and calls **glcm**, which computes the grey-level co-occurrence matrices and their statistics. `grays` sets the quantization level and `window` the moving window; both are glcm's arguments, passed straight through.

```
tex <- analyze_soil_texture(
  rgb,
```

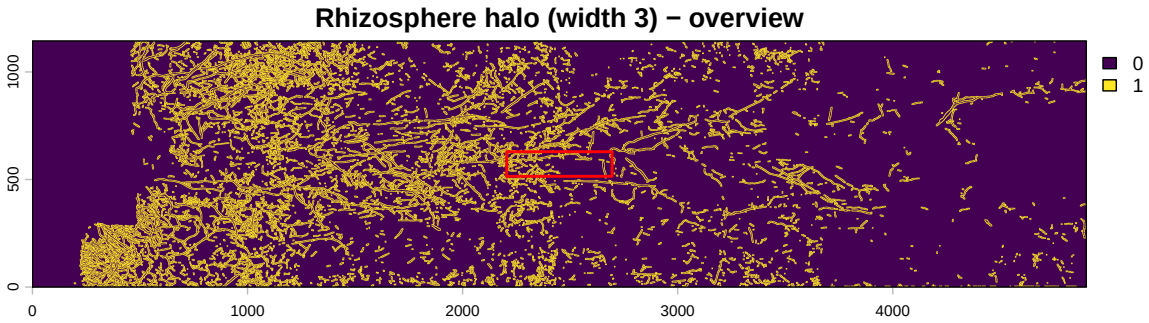
```
  grays   = 12,
  window  = c(5, 5),
  metrics = c("variance", "second_moment", "correlation", "entropy")
)
terra::plot(tex)
```




```
tube_coloration(rgb)
#>       rcc    gcc   bcc        hue saturation luminosity      red    green
#> 1 0.4117 0.3192 0.269 0.06635659  0.1982295    61.0889 67.84946 59.75458
#>       blue
#> 1 54.3997
```

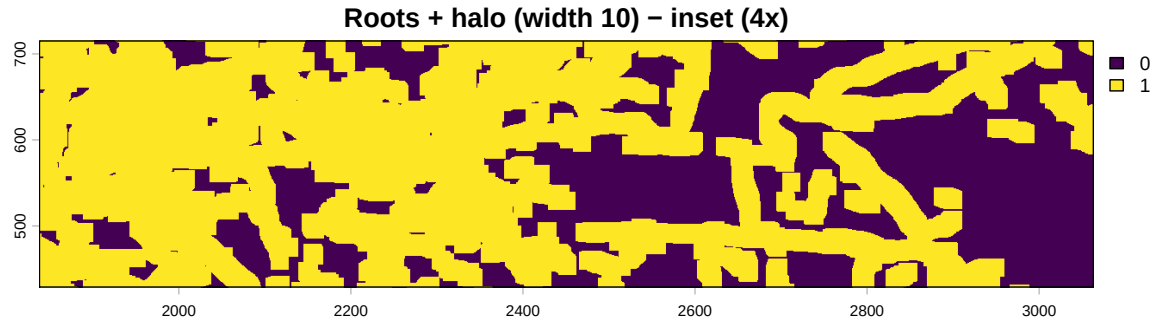
`create_root_buffer()` grows a buffer around every root pixel, a simple proxy for the rhizosphere. With `halo_only = TRUE` it returns the ring alone.

```
halo <- create_root_buffer(root_layer, width = 3, halo_only = TRUE,
                           kernel = "circle")
zoom_plot(halo, main = "Rhizosphere halo (width 3)", frac = 0.1)
```






```
halo10 <- create_root_buffer(root_layer, width = 10, halo_only = FALSE,
                             kernel = "circle")
zoom_plot(halo10, main = "Roots + halo (width 10)", frac = 0.25,
          overview = FALSE)
```



`kernel` switches between a `"circle"` (8-neighbor) and a `"diamond"` (4-neighbor) growth shape, and `width` is the number of dilation iterations.

Flatbed scans: the same traits without a depth axis

A flatbed scan of washed roots is the other geometry the package serves, and it is a special case rather than a different pipeline: the same cleaning, skeleton, length and diameter functions apply, and only the depth axis goes away. Where a root lies on the tray says nothing about where it grew, so the whole scan is one observation.

What that buys is architecture. A washed root system is complete and unoccluded, so the branching graph is worth building on a flatbed scan in a way it rarely is on a tube, where roots enter and leave the field of view.

```
data(flatbed_scan_example)

seg_fb <- load_flexible_image(flatbed_scan_example, output_format = "spatrast",
                              scale = "binary", select_layer = 2)

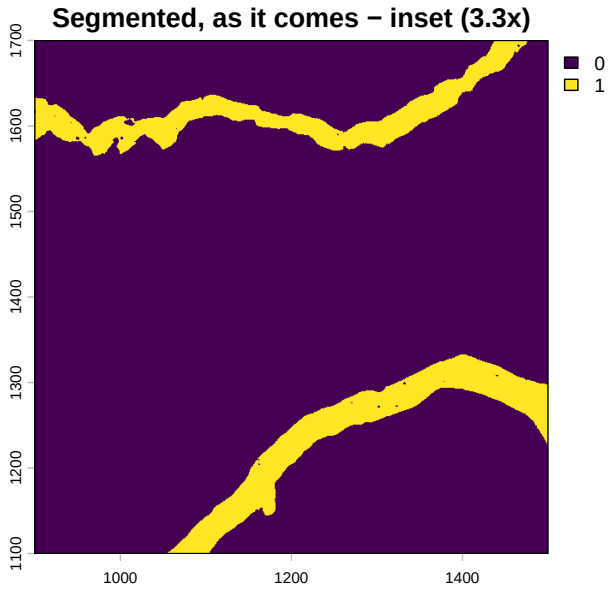
fb_clean <- clean_image(seg_fb,
                        fill_holes_upto   = 50,
                        drop_objects_upto = 50,
                        protect_border    = FALSE)

fb_skl <- skeletonize_image(fb_clean, verbose = FALSE)

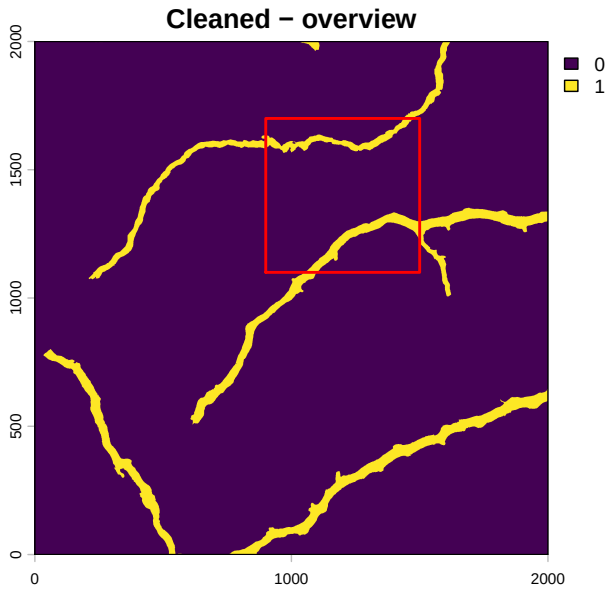
# The order map is built on the unpruned skeleton. Pruning removes the short
# terminal branches, and those are exactly the segments that carry the highest
# orders, so pruning first flattens the very classification being shown.
# `prune_skeleton()` is still the right call before counting tips, and
# `prune_terminal_segments()` prunes inside the graph if you want both.
fb_skl_pruned <- prune_skeleton(fb_skl, mask = fb_clean, min_length = 50,
                                iter = 1)
```

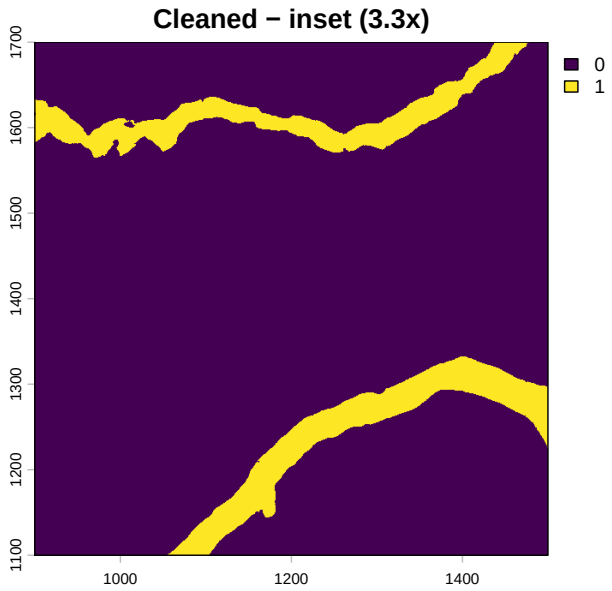
```r
# The segment graph over a whole tray is the slowest step in this article.
order_res <- branch_order_map(
  skel  = fb_skl,
  mask  = fb_clean,          # the mask is what supplies diameters
  order = "order_from_base",
  unit  = "cm",
  dpi   = 1200
)
#> Cropped to 2000x1964 (offset 0,36)
#> Distance transform...
#> Tracing + contracting junctions...
#>   115 segment(s)
#>   spliced pass-through nodes: 115 -> 115 segment(s)
#>   resolved crossings: 115 -> 115 segment(s)
#> Tip order...
#> Done in 0.39s. 115 segments, max order 8, total length 8131.5 px

# The same window before and after cleaning, so the specks that
# drop_objects_upto removes and the pinholes that fill_holes_upto closes can
# be compared rather than described.
zoom_plot(seg_fb,   main = "Segmented, as it comes", center = c(0.6, 0.7),
          overview = FALSE)
```

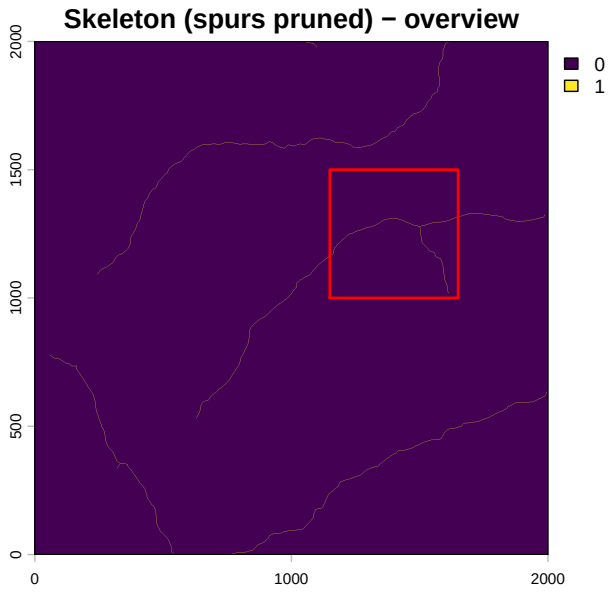


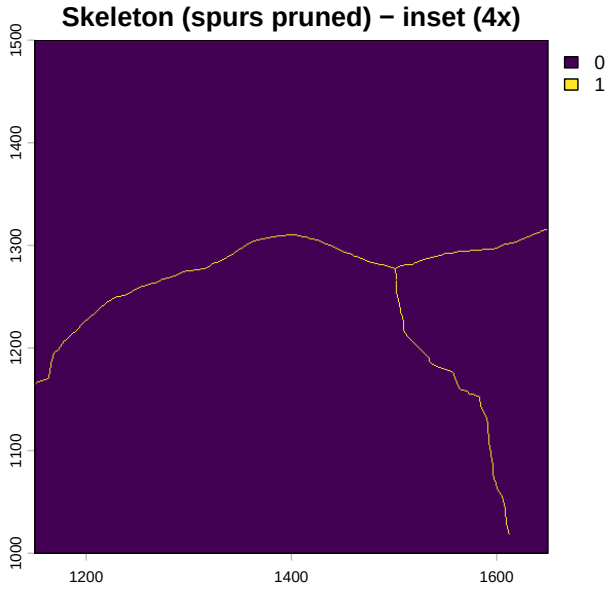

```r
zoom_plot(fb_clean, main = "Cleaned", center = c(0.6, 0.7))
```






```
zoom_plot(fb_skl_pruned, main = "Skeleton (spurs pruned)", frac = 0.25,
          center = c(0.7, 0.625))
```





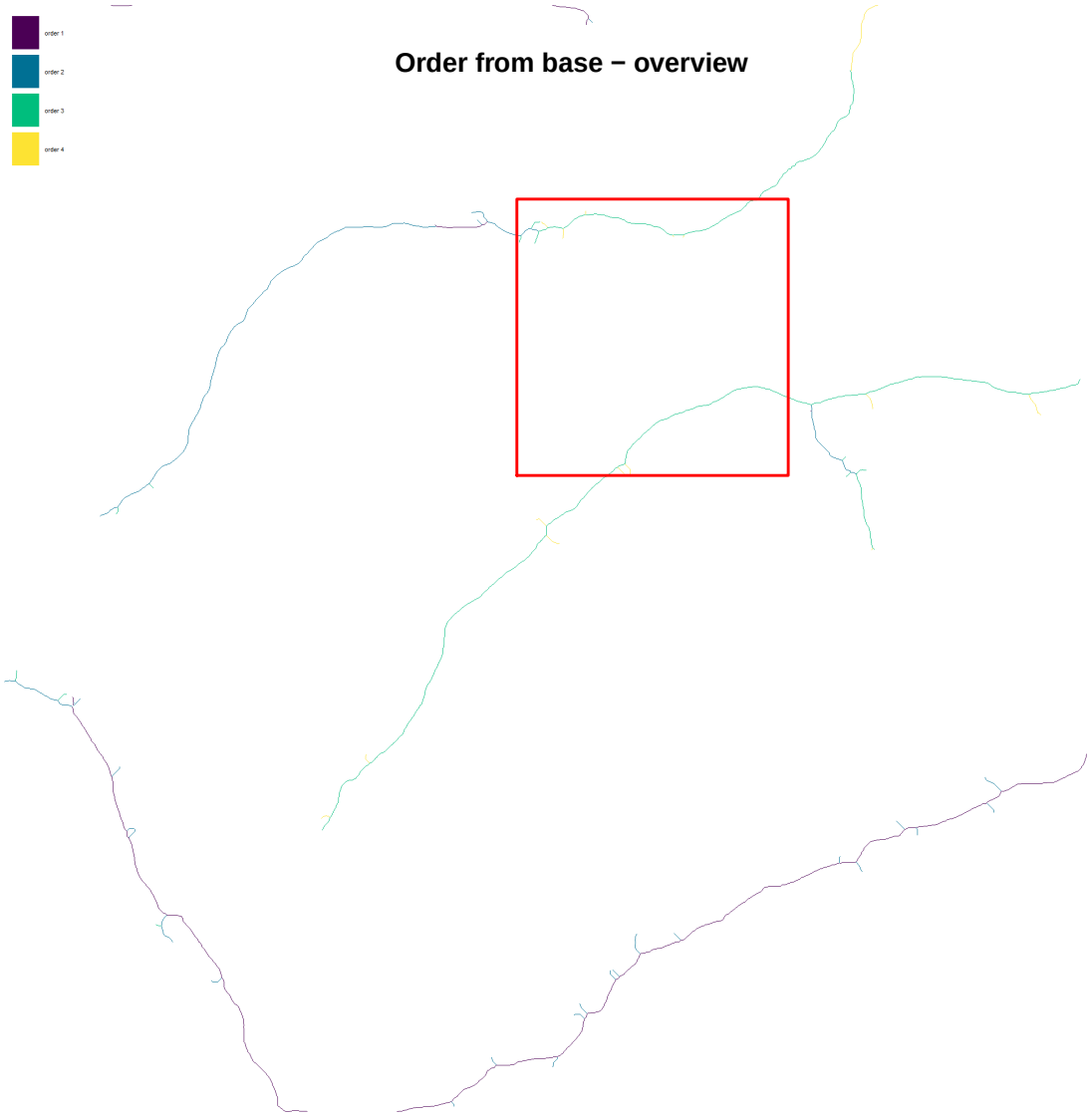
A one-pixel class map plotted whole is unreadable: the lines are a pixel wide and a continuous palette on a white page gives them almost no contrast. `render_order_overlay()` is the function for looking at it – it paints the per-segment value back onto the **mask**, so each root is drawn at its own width, and picks a labeled swatch per class when the values are a handful of small integers.

```
# render_order_overlay() writes the figure with grDevices::png(), which can
# emit a palette image; terra then reads it as a single band of palette
# indices rather than three color bands. png::readPNG() expands the palette,
# so build the raster from that array and zoom_plot() it like any other image.
as_rgb_rast <- function(file) {
  a <- png::readPNG(file)                          # values in 0-1
  if (length(dim(a)) == 2L) a <- array(a, c(dim(a), 3L))
  r <- terra::rast(a[, , 1:3, drop = FALSE] * 255)
  terra::RGB(r) <- 1:3
```

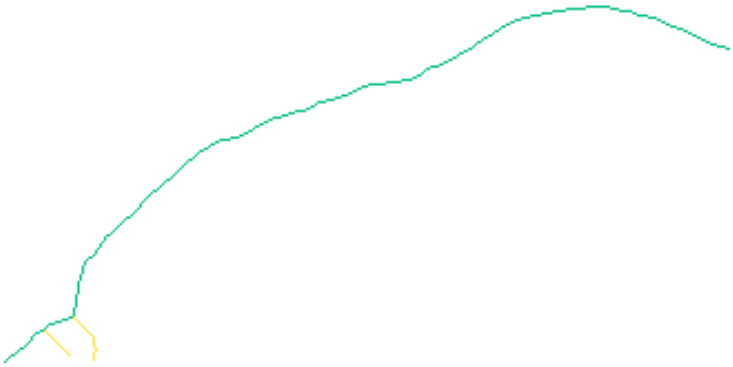
```
  r
}

ord_png <- tempfile(fileext = ".png")
render_order_overlay(attr(order_res$edges, "segments"),
                     order_res$edges$order_from_base,
                     attr(order_res$edges, "dims"), ord_png,
                     mask = fb_clean, value_label = "order from base")

zoom_plot(as_rgb_rast(ord_png), frac = 0.25, center = c(0.6, 0.7), maxcell = Inf,
          main = "Order from base")
```





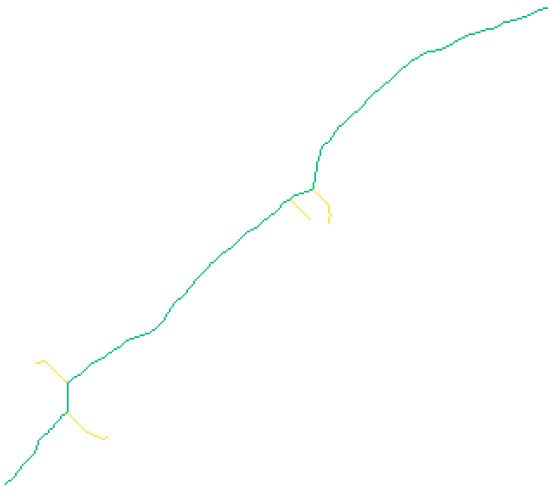


`plot_order_window()` is the one figure that is not a `zoom_plot()`, because it draws two things that are not in any single raster: the skeleton and, over it, the segments the graph actually traced.

To check the tracing itself rather than the orders, `plot_order_window()` renders a chosen rectangle at native resolution and magnified, with no downsampling at all. It draws the skeleton in gray and the traced, ordered segments in color on top, so **bare gray is skeleton the graph missed** – which is the check to run when a number looks wrong.

```
win_png <- tempfile(fileext = ".png")
plot_order_window(order_res$edges, fb_skl,
                  r_range = c(600, 1000), c_range = c(900, 1300),
                  scale = 2, file = win_png,
                  order_col = "order_from_base")

if (requireNamespace("png", quietly = TRUE)) {
  op <- graphics::par(mar = c(0, 0, 0, 0))
  plot(grDevices::as.raster(png::readPNG(win_png)))
  graphics::par(op)
}
```



`order = "order_from_base"` counts generations outward: the thickest, most central path of each connected component is order 1, its branches are 2, theirs 3. Three other schemes are computed at the same time and kept in `order_res$edges`; two count inward from the tips per segment, two describe a whole continuous root. The scheme has to be named because root ecology uses "order" in both directions.

```
order_res$summary
#>   order n_segments n_tips n_branch_points total_length cumulative_length
#> 1     1         31     10              25    5.6299781          5.629978
#> 2     2         39     27              11    3.8839664          9.513944
#> 3     3         30     15              18    6.1129241         15.626869
#> 4     4         15     15               0    0.8279624         16.454831
#>   cumulative_fraction mean_segment_length branching_frequency mean_diameter
#> 1           0.3421474          0.18161220            4.440515    0.06634365
#> 2           0.5781855          0.09958888            2.832156    0.04437175
#> 3           0.9496827          0.20376414            2.944581    0.06009394
#> 4           1.0000000          0.05519749            0.000000    0.04198047
#>   median_diameter
#> 1      0.06806246
#> 2      0.03902940
#> 3      0.04212060
#> 4      0.03611511

# The thickest order class against everything else, picked by diameter rather
# than by order number, so the same call means the same thing across a dataset.
order_metrics(order_res, groups = "thickest")
#>      group orders n_segments n_tips n_branch_points total_length
#> 1 thickest      1         31     10              25     5.629978
#> 2     rest  2,3,4         84     57              29    10.824853
#>   length_fraction mean_segment_length branching_frequency mean_diameter
#> 1       0.3421474           0.1816122            4.440515    0.06634365
#> 2       0.6578526           0.1288673            2.679020    0.05306736
#>   median_diameter
```

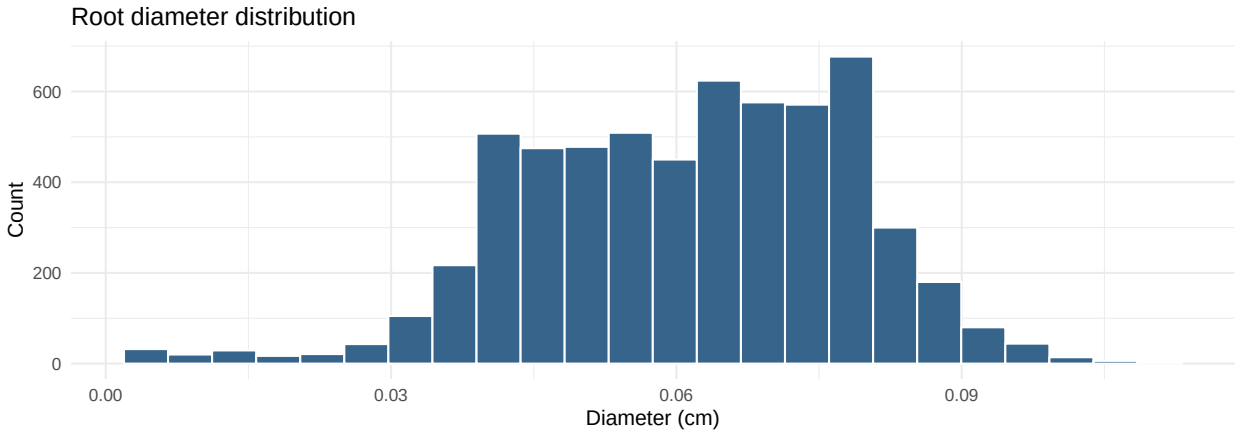
```
#> 1       0.06806246
#> 2       0.03940410
```

Tips and branch points come from this table too: `n_tips` and `n_branch_points` per segment, summed per order class in `$summary`.

```
diam_fb   <- root_diameter(fb_clean, skeleton_img = fb_skl_pruned,
                           unit = "cm", dpi = 1200, select_layer = NULL)
diam_vals <- diam_fb$diameters      # the per-skeleton-pixel values

cat(sprintf("diameter mean %.4f cm | sd %.4f | max %.4f cm\n",
            mean(diam_vals), stats::sd(diam_vals), max(diam_vals)))
#> diameter mean 0.0611 cm | sd 0.0168 | max 0.1108 cm

ggplot(data.frame(diameter = diam_vals), aes(x = diameter)) +
  geom_histogram(bins = 24, fill = "steelblue4", color = "white") +
  theme_minimal() +
  labs(x = "Diameter (cm)", y = "Count",
       title = "Root diameter distribution")
```



A diameter distribution is where fine and coarse roots separate, and `modal_peaks()` finds the modes in it, optionally refining them with a Gaussian mixture via `mclust`. See Flatbed Scans for the full walkthrough.

Network shape and transport

Two questions the segment table answers once it exists, both working on `order_res$edges` rather than on the image.

```
net <- network_metrics(order_res$edges, topology = "projected_tree")
net$traits
#> $n_nodes
#> [1] 121
#>
#> $n_edges
#> [1] 115
#>
#> $n_components
```

```
#> [1] 6
#>
#> $n_tips
#> [1] 67
#>
#> $n_branch_points
#> [1] 54
#>
#> $tip_ratio
#> [1] 0.553719
#>
#> $beta
#> [1] 0.9504132
#>
#> $total_length
#> [1] 16.45483
#>
#> $mean_edge_length
#> [1] 0.1430855
net$qc
#> $cyclomatic
#> [1] 0
#>
#> $alpha_meshedness
#> [1] 0
#>
#> $gamma
#> [1] 0.3221289
#>
#> $n_self_loops
#> [1] 0
#>
#> $n_duplicate_edges
#> [1] 0
#>
#> $is_forest
#> [1] TRUE
#>
#> $artefact_cycles
#> [1] 0
#>
#> $artefact_rate
#> [1] 0
net$note
#> [1] "loops in a projected tree are crossings the segmentation did not separate; artefact_rate divide
```

`$traits` is the shape of the network: node and edge counts, how many separate pieces, how many tips, the tip ratio, and `beta` – edges per node, which is the plainest measure of how branchy a network is. `$qc` holds the loop arithmetic. The same numbers land in one place or the other depending on the `topology` you declare, and nothing else changes:

`topology`	What a loop means	Where loop counts go
`"tree"`	impossible	`$qc`: any loop means the input is not a tree
`"projected_tree"`	crossings in projection	`$qc`: unresolved overlaps
`"reticulate"`	real, e.g. mycelium	`$traits`: a measurement

On a root scan a closed circuit is almost never one the plant grew; it is two roots crossing in front of one another, touching in the image, joined by the skeleton. So `artefact_cycles` counts them and `artefact_rate` divides by total network length, which makes the count comparable between a large root system and a small one. Reporting that as biology is the mistake the argument exists to prevent – which is also why `"reticulate"` is there: for mycelium with anastomoses, leaf venation or a stream network, the loops *are* the structure, and `cyclomatic` and `alpha_meshedness` move into `$traits`.

```
et <- root_transport(order_res$edges)
head(et[, c("length", "path_length", "path_resistance",
            "downstream_tips", "downstream_length")], 4)
#>       length path_length path_resistance downstream_tips downstream_length
#> 1 0.04264472    2.221137         6827712               1        0.04264472
#> 2 0.04009685    2.218590         6929970               1        0.04009685
#> 3 0.18486079    2.178493         6337904               2        0.26760236
#> 4 0.04135668    2.034989         6777683               1        0.04135668
```

`root_transport()` picks the thickest loose end of each separate piece as the base, which on a scan is the proximal end, and measures outward from there. Resistance is the Hagen-Poiseuille term, so it scales with the fourth power of radius: halving a root's diameter makes the same length roughly sixteen times harder to push water through. Treat it as a relative ranking rather than a physical measurement.

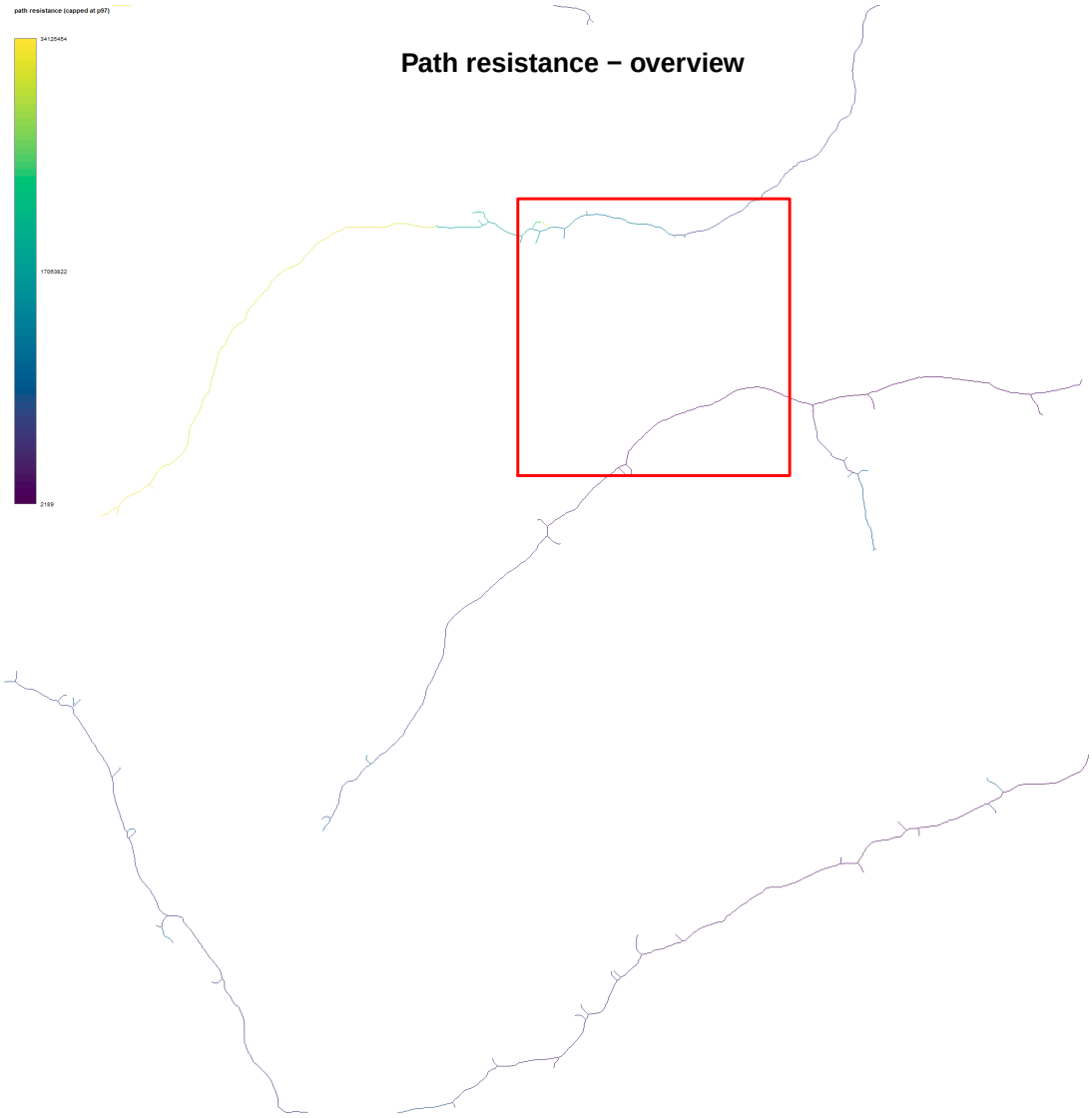
`render_order_overlay()` paints any per-segment column back onto the mask, which is how a transport quantity becomes legible. It picks its own legend: a handful of small integers gets one labeled swatch per class, anything continuous gets a color ramp with a scale bar.

```
png_file <- tempfile(fileext = ".png")

# max_side caps the longest side of the written PNG, and anything below the
# scan's own width throws pixels away: thin roots drop out rather than getting
# thicker. Keep it at the full width and zoom afterwards instead.
# A handful of thin distal segments carry resistance far above the rest, and a
# ramp stretched to that maximum leaves every other segment in nearly the same
# color. Capping the value at the 97th percentile spends the ramp on the range
# the roots actually occupy; the capped segments sit at the top color. The
# alternative is to paint log10(path_resistance) and label the bar as a log
# scale, which keeps the extremes distinguishable but makes the bar harder to
# read as a physical quantity.
cap <- stats::quantile(et$path_resistance, 0.97, na.rm = TRUE)

render_order_overlay(attr(et, "segments"),
                     pmin(et$path_resistance, cap),
                     attr(et, "dims"), png_file,
                     mask = fb_clean, max_side = 2000,
                     value_label = "path resistance (capped at p97)")

zoom_plot(as_rgb_rast(png_file), frac = 0.25, center = c(0.6, 0.7), maxcell = Inf,
          main = "Path resistance")
```


path resistance (capped at p97)
34125454
17063822
2189
Path resistance − overview





Segments with no value are drawn in red, which makes failures visible rather than silently absent – a loop in the graph has no single "beyond", so its transport columns are `NA` and it shows up in that color. A lot of red is the same signal `network_metrics()` reports as `artefact_cycles`.

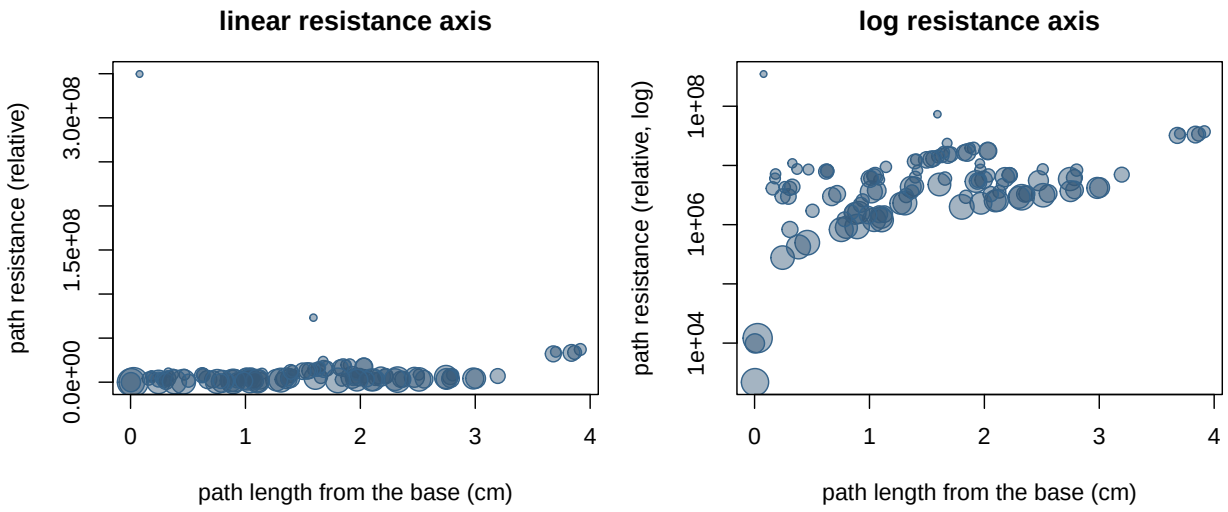
Path length and path resistance rank the segments differently, and the reason is the radius term.

```
op     <- graphics::par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
pt_bg  <- grDevices::rgb(0.21, 0.35, 0.47, 0.45)
pt_cex <- 0.4 + 2.4 * et$mean_diameter / max(et$mean_diameter, na.rm = TRUE)

plot(et$path_length, et$path_resistance,
     pch = 21, bg = pt_bg, col = "steelblue4", cex = pt_cex,
     xlab = "path length from the base (cm)",
     ylab = "path resistance (relative)",
     main = "linear resistance axis")

plot(et$path_length, et$path_resistance, log = "y",
     pch = 21, bg = pt_bg, col = "steelblue4", cex = pt_cex,
     xlab = "path length from the base (cm)",
```

```
     ylab = "path resistance (relative, log)",
     main = "log resistance axis")
```




```
graphics::par(op)
```

The pair is worth seeing. On the linear axis a few thin segments set the top of the range and everything else is squeezed into a band along the bottom, which says only that most segments are cheap and one or two are not. The log axis spreads the same points over the orders of magnitude they actually span, and the structure appears: at any given distance from the base, resistance varies by two or three orders of magnitude. It is the same problem the overlay has, solved there by capping the ramp instead.

Point size is diameter, and the small points – the thin roots – sit at the top. So a long thick root can be an easier path for water than a short thin one, which is why both columns are reported rather than one standing in for the other.

Reference

What you can measure

One row per quantity, with the unit it comes back in and what it says. Lengths, diameters and depths follow `unit` and `dpi`; the table assumes `"cm"`.

Per image, or per depth interval

Quantity	Unit	Meaning
`rootpx`, `voidpx`	pixels	root and background pixel counts
`rootpx.density`	%	root pixels as a share of the interval
`rootlength`	cm	skeleton length, Kimura estimator
`rootlength.density`	cm cm^-2	length per unit of imaged wall
`avg.diameter`, `max.diameter`	cm	from 2x the distance transform
`var.diameter`	cm^2	spread of diameter within the interval
`root.surface.area`	cm^2	lateral area of the root cylinder
`root.volume`	cm^3	volume of the same cylinder
`n.diameter.peaks`	count	modes in the diameter distribution
`rcc`, `gcc`, `bcc`	fraction	chromatic coordinates, sum to 1
`hue`, `saturation`	0-1	HSV of the mean color
`luminosity`	0-255	Rec. 709 weighted mean

Per tube or scan

Quantity	Unit	Meaning
`mrd`	cm	mean rooting depth, amount-weighted
`total.length.density`	cm cm^-2	summed over the profile
`rootlength.fraction`	fraction	share of total length in an interval
`deep_drive`	0-1	share of root pixels heading deepest
`mean.steepness.angle`	degrees	0 horizontal, 90 vertical
Wasserstein distance	cm	depth the mass must travel between profiles
Jensen-Shannon, KL	nats	bin-by-bin divergence, no depth metric
`amplitude`, `offset`	trait units	circumferential sine fit
`peakTime`	slice index	where that sine peaks around the tube

Per segment, from the branching graph

Quantity	Unit	Meaning
`length`	cm	along the pixel chain, junction stub included

Quantity	Unit	Meaning
`mean_diameter`	cm	length-weighted within the segment
`order_from_tip`, `strahler_order`	count	inward from the tips, per segment
`order_from_base`	count	generations outward, per root
`root_max_order_from_tip`	count	the root's largest tip order
`n_tips`, `n_branch_points`	count	degree-1 and degree-3+ nodes
`branching_frequency`	cmˆ-1	branch points per unit length
`path_length`	cm	base of the root system to this segment
`path_resistance`	relative	8 eta L / (pi rˆ4), eta = 1
`downstream_length`	cm	root length beyond this segment
`downstream_tips`	count	tips beyond it
`downstream_surface`	cmˆ2	root surface beyond it

Per network

Quantity	Unit	Meaning
`n_nodes`, `n_edges`	count	graph size
`n_components`	count	separate root systems in the image
`tip_ratio`	fraction	tips over nodes
`beta`	edges/node	how branchy the network is
`cyclomatic`	count	independent closed circuits
`alpha_meshedness`	0-1	loops against the planar maximum
`artefact_cycles`	count	the same, read as unresolved crossings
`artefact_rate`	cmˆ-1	crossings per unit length, comparable

Conventions

Four things are read the same way everywhere, and they are the ones to check before trusting a number.

- **Pixels to physical:** `cm = px * 2.54 / dpi`. Every function that reports a length needs `dpi`, and `root_diameter()` defaults to 300 rather than erroring.
- **Angles:** `angle_from_vertical` and `insertion_angles` are degrees from vertical, `[0, 90]`, 0 upright. `mean.steepness.angle` is degrees from horizontal. They are not the same convention.
- **Axes:** depth runs along the image width, the tube circumference down the height. A scan with the soil surface at the top has to be turned a quarter turn first.
- **Depth zero:** `start_soil` and `soil_starts` are centimeters, subtracted from every depth. `estimate_soil_surface()` reports pixels, so convert.

Two more are easy to misread. Diameter in the order summaries is length-weighted, so `mean_diameter` describes the diameter of the root length rather than the average of segment values. And the Kimura length estimators are not additive, so depth intervals do not sum to the whole-image value.

Packages this builds on

Rootopia is mostly glue and geometry. The numerical work is done by other packages, and knowing which one is behind a function tells you where to look when a result surprises you.

Package	What Rootopia uses it for	Required?
terra	every raster: reading, extents, `zonal()`, plotting	yes
imager	connected components, morphology, the distance transform behind `root_diameter()`	yes
transport	`wasserstein1d()`, the earth-mover distance in `compare_depth_distribution()`	yes
minpack.lm	Levenberg-Marquardt fit in `fit_sine_curve()` / `rhythmicity()`	yes
mclust	Gaussian mixture clustering in `modal_peaks()`	yes
glcm	the texture matrices in `analyze_soil_texture()`	yes
imagefx	phase and cross correlation in `estimate_rotation_shift()`	yes
raster	the object type glcm and RStoolbox expect	yes
dplyr, **tidyr**, **stringr**	shaping the per-interval tables the wrappers return	yes
landscapemetrics	every metric in `root_scape_metrics()`	suggested
RStoolbox	unsupervised classification in `estimate_soil_surface()` and `estimate_rotation_center()`	suggested
png, **tiff**	reading and writing those formats in stitching and overlays	suggested
ggplot2	figures in the vignettes only	suggested

A suggested package is not installed with Rootopia. The functions that need one check for it and stop with a message naming it rather than failing further in, so a thin install still runs the depth-profile workflow in Part I.

The method for `rhythmicity()` is not original here either: it reproduces the likelihood-based rhythmicity tests of diffCircadian (Ding *et al.* 2021, github.com/diffCircadian/diffCircadian) for a spatial axis instead of a temporal one.

Where to go next

- How It Works – the algorithms, the decision rules, and an edge-case table per module.
- Minirhizotron Scans and Flatbed Scans — the step-by-step tutorials.
- Batch Processing — a whole folder in one call.
- Function reference — every exported function.

References

Ding H, Meng L, Liu AC, *et al.* (2021) Likelihood-based tests for detecting circadian rhythmicity and differential circadian patterns in transcriptomic applications. *Briefings in Bioinformatics* 22: bbab224. doi:10.1093/bib/bbab224

Freschet GT, Pagès L, Iversen CM, *et al.* (2021) A starting guide to root ecology. *New Phytologist* 232: 973-1122.

Kimura K, Kikuchi S, Yamasaki S (1999) Accurate root length measurement by image analysis. *Plant and Soil* 216: 117-127.

Peters B, Blume-Werry G, Gillert A, *et al.* (2023) RootDetector: a convolutional neural network for root detection. *Scientific Reports* 13: 1399.

Zhang TY, Suen CY (1984) A fast parallel algorithm for thinning digital patterns. *Communications of the ACM* 27: 236-239.

Software

Entries as `citation()` prints them, with the package versions used here. Where a package names a paper or a book instead of its manual, that is what is cited.

Barthelme S (2024) *imager: Image Processing Library Based on 'CImg'*. R package version 1.0.2.

Elzhov TV, Mullen KM, Spiess A, Bolker B (2023) *minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm Found in MINPACK, Plus Support for Bounds*. R package version 1.2-4.

Hesselbarth MHK, Sciaini M, With KA, Wiegand K, Nowosad J (2019) landscapemetrics: an open-source R tool to calculate landscape metrics. *Ecography* 42: 1648-1657.

Hijmans R (2025) *raster: Geographic Data Analysis and Modeling*. R package version 3.6-31.

Hijmans R, Brown A, Barbosa M (2026) *terra: Spatial Data Analysis*. R package version 1.9-11.

Müller K, Schwalb-Willmann J, Wegmann M, Leutner B (2024) RStoolbox: an R package for remote sensing data analysis. *Methods in Ecology and Evolution*. doi:10.1111/2041-210X.14451

Schuhmacher D, Bähre B, Bonneel N, Gottschlich C, Hartmann V, Heinemann F, Schmitzer B, Schrieber J (2024) *transport: Computation of Optimal Transport Plans and Wasserstein Distances*. R package version 0.15-4.

Scrucca L, Fraley C, Murphy TB, Raftery AE (2023) *Model-Based Clustering, Classification, and Density Estimation Using mclust in R*. Chapman and Hall/CRC. doi:10.1201/9781003277965

Urbanek S (2022) *png: Read and write PNG images*. R package version 0.1-8.

Urbanek S, Johnson K (2023) *tiff: Read and Write TIFF Images*. R package version 0.1-12.

Wickham H (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag, New York.

Wickham H (2025) *stringr: Simple, Consistent Wrappers for Common String Operations*. R package version 1.6.0.

Wickham H, François R, Henry L, Müller K, Vaughan D (2023) *dplyr: A Grammar of Data Manipulation*. R package version 1.1.4.

Wickham H, Vaughan D, Girlich M (2024) *tidyr: Tidy Messy Data*. R package version 1.3.1.

Witsil AJ (2020) *imagefx: Extract Features from Images*. R package version 0.4.1.

Zvoleff A (2020) *glcm: Calculate Textures from Grey-Level Co-Occurrence Matrices (GLCMs)*. R package version 1.6.5.